# Real-Time Smart Parking System Using AI, Cloud Computing, and IoT

# 1. ABSTRACT

# 1.1.PROBLEM DEFINITION

Urban areas are experiencing rapid growth in vehicle usage, leading to significant challenges in parking management. Conventional parking systems are largely inefficient, relying on manual monitoring or static infrastructure that lacks real-time visibility. This results in increased traffic congestion, fuel wastage, time loss, and driver frustration due to the inability to quickly locate available parking spaces. Furthermore, existing systems often lack intelligent decision-making capabilities, seamless digital payment integration, and centralized data management.

This project, titled **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT,"** addresses these limitations by proposing an integrated and automated parking solution. The system utilizes IoT-based sensors to detect the real-time occupancy status of parking slots and continuously updates this data to a cloud-based platform. Leveraging artificial intelligence, the system analyzes availability patterns and provides optimized parking slot recommendations to users. Additionally, the integration of a user-friendly interface and digital payment mechanism enhances the overall user experience and operational efficiency.

By combining IoT for data acquisition, cloud computing for real-time data synchronization, and AI for intelligent decision-making, the proposed system offers a scalable and efficient solution to modern parking problems. The implementation demonstrates improved space utilization, reduced congestion, and enhanced convenience, making it suitable for deployment in smart cities and urban infrastructures.

# 2. INTRODUCTION

# 2.1.INTRODUCTION

The rapid growth of urbanization and the continuous increase in the number of vehicles have created significant challenges in managing parking spaces efficiently. In modern cities, finding an available parking slot has become a time-consuming and frustrating task for drivers. Conventional parking systems are largely manual or semi-automated, lacking real-time monitoring and intelligent decision-making capabilities. As a result, vehicles often circulate unnecessarily in search of parking, leading to increased traffic congestion, fuel consumption, environmental pollution, and overall inefficiency in transportation systems.

With the advancement of technology, the concept of smart cities has emerged, emphasizing the use of digital solutions to improve urban infrastructure and services. One critical component of smart city development is intelligent parking management. A smart parking system aims to automate the process of detecting, monitoring, and managing parking spaces while providing real-time information to users. This not only enhances user convenience but also contributes to better traffic flow and optimized utilization of available parking resources.

The proposed project, titled **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT,"** focuses on developing an integrated solution that addresses the limitations of traditional parking systems. The system utilizes Internet of Things (IoT) sensors to detect the presence or absence of vehicles in parking slots. These sensors continuously collect data and transmit it to a cloud-based platform, ensuring real-time updates and centralized data management. Cloud computing enables scalability, reliability, and accessibility of the system across different devices and locations.

Artificial Intelligence (AI) plays a key role in enhancing the functionality of the system by analyzing real-time and historical data to provide intelligent parking slot recommendations. This helps users quickly identify the most suitable parking space, thereby reducing search time and improving overall efficiency. Additionally, the system includes a user-friendly interface that allows users to view parking availability, make bookings, and complete digital payments seamlessly.

The integration of IoT, cloud computing, and AI creates a robust and efficient parking management system that is capable of adapting to dynamic urban environments. By automating the parking process and providing real-time insights, the proposed system reduces human effort, minimizes errors, and enhances the overall user experience. Furthermore, it supports sustainable urban development by reducing unnecessary vehicle movement and lowering carbon emissions.

In conclusion, the development of a real-time smart parking system is a necessary step toward addressing the growing parking challenges in urban areas. This project demonstrates how modern technologies can be effectively combined to create an intelligent, scalable, and user-centric solution that aligns with the goals of smart city initiatives.

# 3. SYSTEM ANALYSIS AND STUDY

# 3.1.EXISTING SYSTEM

The current parking systems used in most urban areas are either manual or semi-automated and lack real-time intelligence. In traditional systems, drivers are required to physically search for available parking spaces, often relying on visual inspection or assistance from parking attendants. This process is inefficient, time-consuming, and leads to increased traffic congestion, especially in high-density areas such as shopping centers, offices, and public institutions.

Some existing systems incorporate basic automation, such as entry/exit ticketing or static display boards indicating slot availability. However, these systems are limited in functionality as they do not provide real-time updates at the individual slot level. They also lack integration with modern technologies such as cloud computing and mobile interfaces, making remote access and monitoring difficult. Additionally, there is no intelligent mechanism to guide users to the nearest or most optimal parking space.

Another major limitation of existing systems is the absence of centralized data management. Parking data is often not stored or analyzed effectively, which prevents the use of predictive analytics or optimization techniques. Payment processes in many systems are still manual or require separate infrastructure, leading to inconvenience and inefficiency. Furthermore, these systems are prone to human errors and do not scale well with increasing demand.

# 3.2.PROPOSED SYSTEM

The proposed system, titled **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT,"** provides an advanced and automated solution to overcome the limitations of traditional parking systems. It uses IoT-based sensors to detect vehicle presence in each parking slot and updates the status in real time. This data is transmitted to a cloud-based platform, ensuring continuous synchronization and centralized data management.

Artificial Intelligence is integrated into the system to analyze parking data and provide smart recommendations to users, helping them quickly identify the most suitable parking space. A user-friendly interface allows users to view real-time availability, book slots, and make secure digital payments. The system reduces manual intervention, improves efficiency, and enhances user experience.

## Key Features of Proposed System :

- Real-time parking slot detection using IoT sensors
- Cloud-based data storage and synchronization
- AI-based parking slot recommendation
- User-friendly interface for easy access and booking
- Integrated digital payment system
- Reduction in traffic congestion and search time
- Efficient utilization of parking space
- Scalable design suitable for smart city implementation
- Improved accuracy and reduced human errors

Overall, the proposed system offers a reliable, intelligent, and scalable parking management solution that significantly improves upon existing methods and aligns with modern technological advancements.

# 3.3.MODULE DESCRIPTION

The proposed **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT"** is divided into several functional modules. Each module is responsible for a specific operation, ensuring that the overall system works efficiently, accurately, and in real time.

## 1. User Module

The User Module serves as the interface between the system and the end user. It allows users to interact with the system through a web or mobile application.

**Functions:**

- User registration and login
- Viewing real-time parking availability
- Booking/reserving parking slots
- Receiving AI-based slot recommendations
- Making digital payments

This module ensures ease of use and provides a smooth user experience.

## 2. Parking Management Module

This module manages all parking-related operations and maintains the status of each parking slot.

**Functions:**

- Monitoring slot availability (occupied/vacant)
- Updating slot status in real time
- Allocating and deallocating parking slots
- Managing parking layout and slot details
  It acts as the core system that controls parking space utilization.

## 3. IoT Sensor Module

This module is responsible for detecting the presence or absence of vehicles in parking slots using sensors.

**Functions:**

- Detecting vehicle entry and exit
- Sending real-time data to the cloud
- Ensuring accurate slot status updates
- Typically uses:
- Ultrasonic sensors or IR sensors
- Microcontrollers like Arduino or ESP 32

This module enables real-time data collection.

## 4. Cloud (Firebase) Module

The cloud module handles data storage, synchronization, and communication between system components.

**Functions:**

- Storing parking slot data and user information
- Real-time database updates
- Synchronizing data between sensors and UI
- Handling authentication and backend logic

This ensures scalability, reliability, and remote access.

## 5. AI Recommendation Module

This module adds intelligence to the system by analyzing parking data and suggesting optimal slots.

**Functions:**

- Analyzing real-time and historical data
- Recommending nearest or best available slot
- Reducing user search time

Improving parking efficiency

## 6. Payment Module (Demo Implementation)

The Payment Module in the proposed system is implemented as a demonstration module to simulate the functionality of digital transactions. It is designed to represent how an actual payment system would operate in a real-world smart parking environment.

**Functions:**

- Displaying parking fee based on duration
- Simulating payment process (no real transaction)
- Generating payment confirmation message
- Updating booking/payment status in the system

Unlike fully deployed systems, this module does not integrate with real payment gateways. Instead, it provides a mock interface to demonstrate the workflow of online payment, ensuring that the overall system design remains complete and understandable.

**Reason for Demo Implementation:**

- Avoids dependency on third-party payment APIs
- Simplifies development and testing
- Suitable for academic project demonstration

**Future Enhancement:**

- Integration with real payment gateways (e.g., UPI, credit/debit cards)
- Secure transaction handling and encryption
- Automated billing and invoice generation

## 7. Staff (Admin) Module

The Staff Module is responsible for monitoring, managing, and maintaining the overall parking system. It provides administrative control over parking operations and ensures smooth functioning of all system components.

**Functions:**

- Monitoring real-time parking slot status
- Managing parking slots (add/remove/update slots)
- Viewing user bookings and activity
- Verifying and overseeing payment status (demo-based)
- Handling system errors or anomalies
- Managing user accounts (optional)

This module acts as the control center of the system, allowing authorized personnel to supervise and manage operations efficiently.

# 3.4.FEASIBILITY STUDY

A feasibility study is conducted to evaluate the practicality and effectiveness of the proposed **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT."** It analyzes whether the system can be successfully developed and implemented within given constraints. The study is categorized into technical, economic, and operational feasibility.

## 1. Technical Feasibility

Technical feasibility assesses whether the required technology, tools, and resources are available to develop and implement the system.

The proposed system uses widely available and well-supported technologies such as IoT sensors (ultrasonic/IR), microcontrollers (Arduino/ESP-32), cloud platforms (Firebase), and web-based user interfaces. These technologies are compatible with each other and are suitable for real-time data processing and communication. Firebase provides reliable backend services including real-time database, authentication, and cloud hosting, making system integration efficient.

The AI component is implemented using basic rule-based logic, which does not require high computational resources and can be easily integrated into the system. The development tools and programming environments required are also readily accessible.

Overall, the system is technically feasible as it relies on proven technologies, requires moderate system resources, and can be implemented within an academic environment.

## 2. Economic Feasibility

Economic feasibility evaluates the cost-effectiveness of the system and whether the benefits outweigh the development and operational costs.

The proposed system is cost-efficient as it uses low-cost hardware components such as sensors and microcontrollers. Cloud services like Firebase offer free or low-cost tiers suitable for prototype development. Since the payment module is implemented as a demo, there are no additional costs related to payment gateway integration.

Development costs are minimal as open-source tools and technologies are used. Maintenance costs are also relatively low, as the system does not require complex infrastructure.

The benefits of the system, such as reduced traffic congestion, time savings, efficient parking management, and improved user convenience, justify the investment. Therefore, the system is economically feasible, especially for small to medium-scale implementations.

**3. Operational Feasibility**

Operational feasibility determines whether the system can be effectively used and managed in a real-world environment.

The system is designed with a user-friendly interface, making it easy for users to check parking availability, book slots, and perform other operations without technical difficulty. Real-time updates ensure accurate information, improving user trust and usability.

Administrative control can be managed through the backend (Firebase), allowing staff to monitor and maintain the system efficiently. Although the payment module is a demo, it successfully demonstrates the workflow and can be upgraded to a real payment system in the future.

The system reduces manual effort, minimizes human errors, and improves overall parking efficiency. It can be easily adapted and scaled for larger environments such as malls, offices, and smart cities.

Thus, the system is operationally feasible as it meets user requirements, is easy to use, and can be effectively maintained and expanded.

## Conclusion of Feasibility Study

Based on technical, economic, and operational analysis, the proposed smart parking system is feasible and practical. It can be successfully implemented using available technologies, within reasonable cost, and with efficient operational performance.

# 3.5.TIMELINE CHART

| SYSTEM ANALYSIS | STARTING DATE | ENDING DATE |
|---|---|---|
| SYSTEM STUDY | 01/02/2026 | 05/02/2026 |
| REQUIREMENT ANALYSIS | 06/02/2026 | 10/02/2026 |

| SYSTEM DESIGN | | |
|---|---|---|
| **HIGHLEVEL DESIN** | **STARTING DATE** | **ENDING DATE** |
| STRUCTURED CHART | 11/02/2026 | 15/02/2026 |
| ER DIAGRAM | 16/02/2026 | 18/02/2026 |

| DATABASE DESIGN | STARTING DATE | ENDING DATE |
|---|---|---|
| PHYSICAL DATABASE DESIGN | 19/02/2026 | 20/02/2026 |
| DFD | 20/02/2026 | 23/02/2026 |
| TABLE DESIGN | 23/02/2026 | 24/02/2026 |

| DESIGN PHASE | STARTING DATE | ENDING DATE |
|---|---|---|
| UI DESIGN | 28/02/2026 | 01/03/2026 |
| CODING | 01/03/2026 | 07/03/2026 |
| HARDWARE | 08/03/2026 | 18/03/2026 |

| TESTING AND IMPLIMENTATION | STARTING DATE | ENDING DATE |
|---|---|---|
| UNIT TESTING | 19/03/2026 | 22/03/2026 |
| INTEGRATION TESTING | 23/03/2026 | 27/03/2026 |
| VALIDATION TESTING | 28/03/2026 | 01/04/2026 |
| OUTPUT TESTING | 02/04/2026 | 18/04/2026 |
| IMPLEMENTATION | 19/04/2026 | 29/04/2026 |
| USER ACCEPTANCE TEST | 01/05/2026 | 04/05/2026 |

# 4. SYSTEM REQUIREMENTS

# 4.1.HARDWARE SPECIFICATIONS

The hardware requirements for the proposed “Real-Time Smart Parking System Using AI, Cloud Computing, and IoT” are categorized based on development, sensing, processing, and deployment needs. The system is designed to operate using cost-effective and easily available hardware components.

## 1. Development System (Client Machine)

This is the system used for designing, coding, and testing the application.

**Specifications:**

- Processor: Intel Core i5 (9th Gen or above) / AMD equivalent
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB SSD
- Operating System: Windows 10/11 / Linux / macOS
- Internet Connectivity: Required for Firebase and cloud access

## 2. IoT Sensor Hardware

Used for detecting vehicle presence in parking slots.

**Components:**

- Ultrasonic Sensors (HC-SR04) or IR Sensors

Detect distance or object presence, installed in each parking slot

## 3. Microcontroller Unit

Acts as the interface between sensors and the cloud.

**Options:**

- Arduino Uno
- NodeMCU (ESP8266) / ESP32 (preferred for Wi-Fi capability)

**Functions:**

- Collect data from sensors
- Process input signals
- Transmit data to cloud (Firebase)

**4. Communication Module**

Required for transmitting sensor data to the cloud.

- Built-in Wi-Fi (ESP8266 / ESP32)
- Alternatively: External Wi-Fi module (ESP8266 with Arduino)

**5. Power Supply**

Provides power to sensors and microcontroller.

**Requirements:**

- 5V regulated power supply
- Battery or adapter-based power source
- Optional: Power bank for portable setup

**6. Parking Slot Setup Hardware**

Physical setup for demonstration.

- Prototype parking model (board or frame)
- LED indicators (optional for slot status)
- Connecting wires and breadboard

**7. User Interface Device**

Used by users to access the system.

- Smartphone / Laptop / Desktop

# 4.2.SOFTWARE SPECIFICATIONS

- **Operating System** : Windows 10/11 (64-bit platform)
- **Front End** : HTML, CSS, JavaScript
- **Back End** : Firebase (Cloud-based Backend)
- **Database** : Firebase Realtime Database / Cloud Firestore
- **Programming Languages** :
  - JavaScript (Frontend & Logic)
  - Embedded C (Arduino Programming for IoT)
- **IDE / Development Tools** :
  - Visual Studio Code (VS Code)
  - Arduino IDE
- **Cloud Platform** : Firebase (Authentication, Hosting, Database)
- **Browser** : Google Chrome / Microsoft Edge
- **IoT Software Support** : Arduino Libraries for sensor and Wi-Fi communication

# 4.3.OPERATING SYSTEM

**Windows 11:**

Windows 11 is used as the primary operating system for the development and execution of the proposed smart parking system. It provides a modern, user-friendly interface along with enhanced performance, security, and compatibility features. The operating system supports all the required development tools such as Visual Studio Code, Arduino IDE, and web browsers like Google Chrome and Microsoft Edge, which are essential for building and testing the application. Windows 11 also offers efficient multitasking capabilities, stable network connectivity for cloud-based services like Firebase, and seamless driver support for IoT hardware components. Its robust architecture and regular updates ensure a secure and reliable environment for both development and deployment of the system.

# 4.4.TECHNOLOGY

**HTML (HyperText Markup Language):**

HTML is used to structure the web pages of the smart parking system. It defines the layout of the user interface, including elements such as buttons, forms, parking slot displays, and navigation components. It acts as the backbone of the frontend.

**CSS (Cascading Style Sheets):**

CSS is used to design and enhance the visual appearance of the application. It controls layout, colors, fonts, spacing, and responsiveness, ensuring that the user interface is clean, user-friendly, and visually appealing.

**JavaScript:**

JavaScript is used to implement the core functionality of the system on the frontend. It handles dynamic updates such as real-time slot status display, user interactions, booking logic, and integration with Firebase. It also powers the AI-based recommendation logic in the system.

**Firebase:**

Firebase is used as the backend platform for the system. It provides real-time database services, authentication, and cloud hosting. Firebase ensures that parking slot data is updated instantly and can be accessed from anywhere, enabling real-time synchronization between IoT devices and the user interface.

**Firebase Realtime Database / Firestore:**

This is used to store and manage application data such as parking slot status, user information, and booking details. It allows real-time updates, ensuring that users always see the latest parking availability.

**Arduino IDE:**

Arduino IDE is used to write, compile, and upload code to the microcontroller. It enables the programming of IoT components such as sensors and handles communication between hardware and the cloud.

**Embedded C (Arduino Programming):**

Embedded C is used to program the microcontroller (Arduino/NodeMCU). It controls the IoT sensors, processes input signals, and sends parking slot data to the cloud in real time.

**IoT Sensors (Ultrasonic / IR Sensors):**

These sensors are used to detect the presence or absence of vehicles in parking slots. They provide real-time data to the system, enabling accurate monitoring of parking availability.

**ESP32 / ESP8266 (Microcontroller):**

This microcontroller acts as the interface between sensors and the cloud. It processes sensor data and transmits it to Firebase using Wi-Fi, enabling real-time communication.

**Web Browser (Google Chrome / Edge):**

Web browsers are used to run and test the application. They provide the interface through which users access the system, view parking availability, and perform booking operations.

**AI Logic (Rule-Based System):**

The AI component in the system is implemented using rule-based logic. It analyzes parking slot data and suggests the most suitable or nearest available slot to the user, improving efficiency and reducing search time.

# 5. SYSTEM DESIGN AND DEVELOPMENT PROCESS

# 5.1.SYSTEM DESIGN

## 1. Architectural Design

The proposed **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT"** follows a **layered architecture** that integrates IoT devices, cloud services, and user interfaces for real-time operation.

**Architecture Layers:**

a) IoT Layer

- Consists of sensors (Ultrasonic/IR) installed in parking slots
- Detects vehicle presence (occupied/vacant)
- Sends data to microcontroller

b) Processing Layer (Microcontroller)

- NodeMCU / ESP8266 processes sensor input
- Converts physical signals into digital data
- Sends data to cloud via Wi-Fi

c) Cloud Layer

- Firebase acts as backend
- Stores parking data, user details, booking info
- Provides real-time synchronization

d) Application Layer (Frontend)

- Web interface using HTML, CSS, JavaScript
- Displays parking availability
- Allows booking and payment (demo)

e) AI Layer

- Applies rule-based logic
- Recommends best available parking slot

- Optimizes user decision-making

**Architecture Flow:**

Sensor → Microcontroller → Firebase (Cloud) → Web UI → AI Recommendation → User Action

## 2. Design Patterns Used

a) MVC (Model-View-Controller) Pattern

The system loosely follows the MVC architecture:

- **Model** : Firebase Database (stores parking data)
- **View** : User Interface (HTML, CSS)
- **Controller** : JavaScript logic handling interactions

**Benefit:** Separates logic, data, and UI

Makes system easier to maintain and scale

b) Client-Server Architecture

- Client: User interface (browser)
- Server: Firebase backend

**Benefit:**

Enables real-time communication

Supports multiple users simultaneously

c) Publish-Subscribe Pattern (Firebase)

- Sensors update data → Firebase
- UI automatically receives updates

**Benefit:**

Real-time synchronization

No need for manual refresh

## 3. Class and Sequence Diagrams

• **Class Diagrams:** Illustrating classes such as User, ParkingSlot, Booking, Payment, Admin, and IoTDevice with their relationships and attributes. These classes represent the core components of the smart parking system, including user interactions, slot management, booking details, and system administration.

• **Sequence Diagrams:** Depicting interactions between system components for use cases such as viewing parking availability, slot booking, AI-based recommendation, and payment processing (demo). These diagrams show the step-by-step flow of data between the user interface, cloud database (Firebase), AI module, and IoT devices.

## 4.Modules

### 1. User Module

• User registration and login
• View real-time parking availability
• Book/reserve parking slots
• Receive AI-based slot recommendations
• Make payment (demo)
• View booking details

**2. Parking Management Module**

- Monitor parking slot status (occupied/vacant)
- Update slot availability in real time
- Allocate and deallocate parking slots
- Maintain parking layout details

**3. IoT Sensor Module**

- Detect vehicle presence using sensors
- Send real-time data to microcontroller
- Ensure accurate slot status updates

**4. Cloud (Firebase) Module**

- Store parking and user data
- Real-time database synchronization
- Handle backend operations
- Enable remote access

**5. AI Recommendation Module**

- Analyze parking data
- Suggest best available slot
- Reduce search time
- Improve efficiency

**6. Payment Module (Demo)**

- Display parking charges
- Simulate payment process

• Generate confirmation message
• Update payment status

**7. Admin (Staff) Module**

• Monitor system activity
• Manage parking slots
• View user bookings
• Handle system updates

## 5. Database Design

• **Entity-Relationship Diagram (ERD):** Designing the database schema to represent entities such as User, ParkingSlot, Booking, Payment, Admin, and IoTDevice, along with their relationships for efficient parking management and data handling.

• **Normalization:** Ensuring data integrity and minimizing redundancy by organizing data into structured tables (e.g., separating user, booking, and slot data) using normalization techniques.

• **Indexing and Query Optimization:** Improving database performance by efficiently structuring data in Firebase for faster retrieval, real-time updates, and optimized query handling.

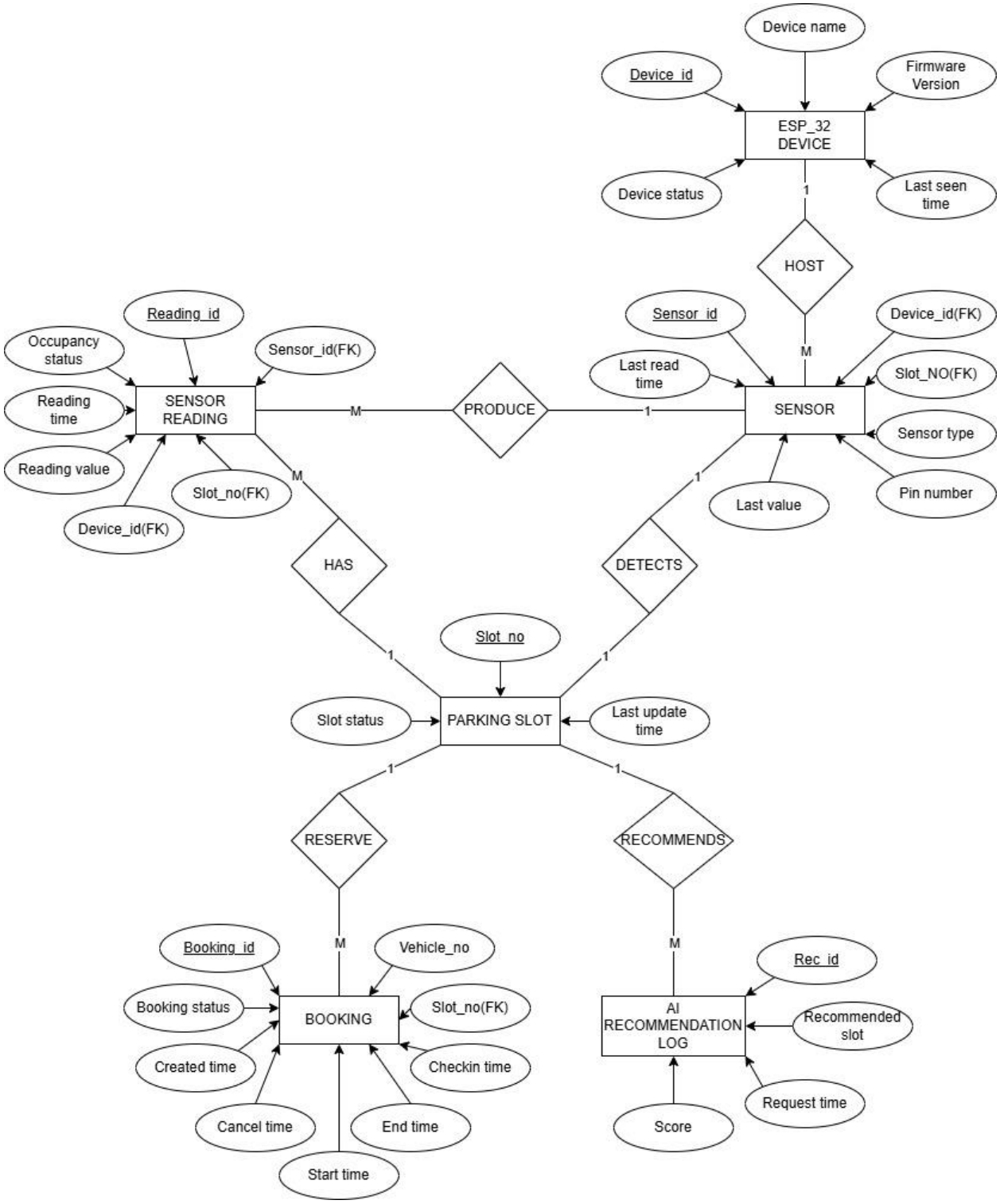
# 5.2. ENTITY RELATIONSHIP DIAGRAM

The graphical representation of overall logical structure of a database is called ER diagram. This diagram displays the inter relationship of the entity sets and their attributes. This is a relatively simple and clear representation of a database. The following figure shows the entity sets include relationships participation and the nature of entity set etc. Symbols used in ER model:

- Rectangle: Rectangles represent Entities in the ER Model.
- Ellipse: Ellipses represent Attributes in the ER Model.
- Diamond: Diamonds represent Relationships among Entities.
- Line: Lines represent attributes to entities and entity sets with other relationship types.
- Double Ellipse: Double Ellipses represent Multi-Valued Attributes.
- Underlined Ellipse : Represent primary key.

| Symbol Name | Symbol | Represents |
|---|---|---|
| Rectangles | | Represents Entity |
| Ellipses | | Represents Attribute |
| Diamonds | | Represents Relationship |
| Lines | | Links Attribute(s) to entity set(s) or Entity set(s) to Relationship set(s) |
| Double Ellipses | | Represents Multivalued Attributes |
| Primary key | | Represents Key Attributes / Single Valued Attributes |

# ER DIAGRAM :

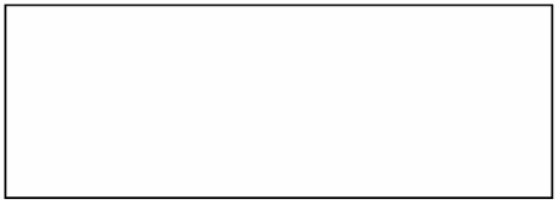

Device name
Device_id
Firmware Version
ESP_32 DEVICE
Device status
Last seen time
1
HOST
M
Reading_id
Occupancy status
Sensor_id(FK)
Reading time
SENSOR READING
Reading value
Slot_no(FK)
Device_id(FK)
M
PRODUCE
1
Sensor_id
Last read time
SENSOR
Device_id(FK)
Slot_NO(FK)
Sensor type
Pin number
Last value
M
1
HAS
DETECTS
1
1
Slot_no
Slot status
PARKING SLOT
Last update time
1
1
RESERVE
RECOMMENDS
M
M
Booking_id
Vehicle_no
Booking status
BOOKING
Slot_no(FK)
Created time
Checkin time
Cancel time
End time
Start time
Rec_id
AI RECOMMENDATION LOG
Recommended slot
Request time
Score

# 5.3.DATAFLOW DIAGRAM

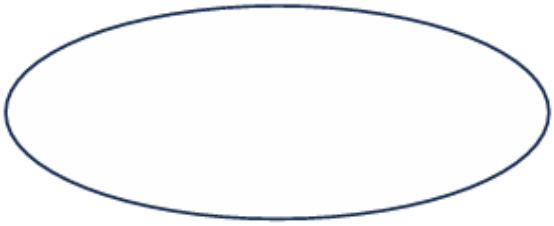
Data flow diagrams are used to graphically represent the flow of data in a business information system. DFD describes the processes that are involved in a system to transfer data from the input to the file storage and reports generation.

## DFD NOTATIONS

**External Entity**: An outside system that sends or receives data, communicating with the system being diagrammed. They are the sources and destinations of information entering or leaving the system. They might be an outside organization or person, a computer system or a business system. They are also known as terminators, sources and sinks or actors. They are typically drawn on the edges of the diagram.

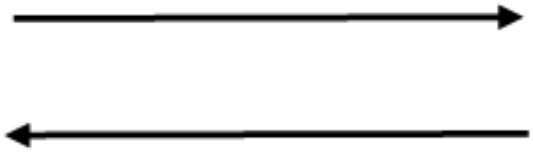


**Process:** Any process that changes the data, producing an output. It might perform computations, or sort data.



**Data Store**: Files or repositories that hold information for later use, such as a database table or a membership.

**Data Flow**: The route that data takes between the external entities, processes and data stores. It portrays the interface between the other components and is shown with arrows, typically labelled with a short data name.

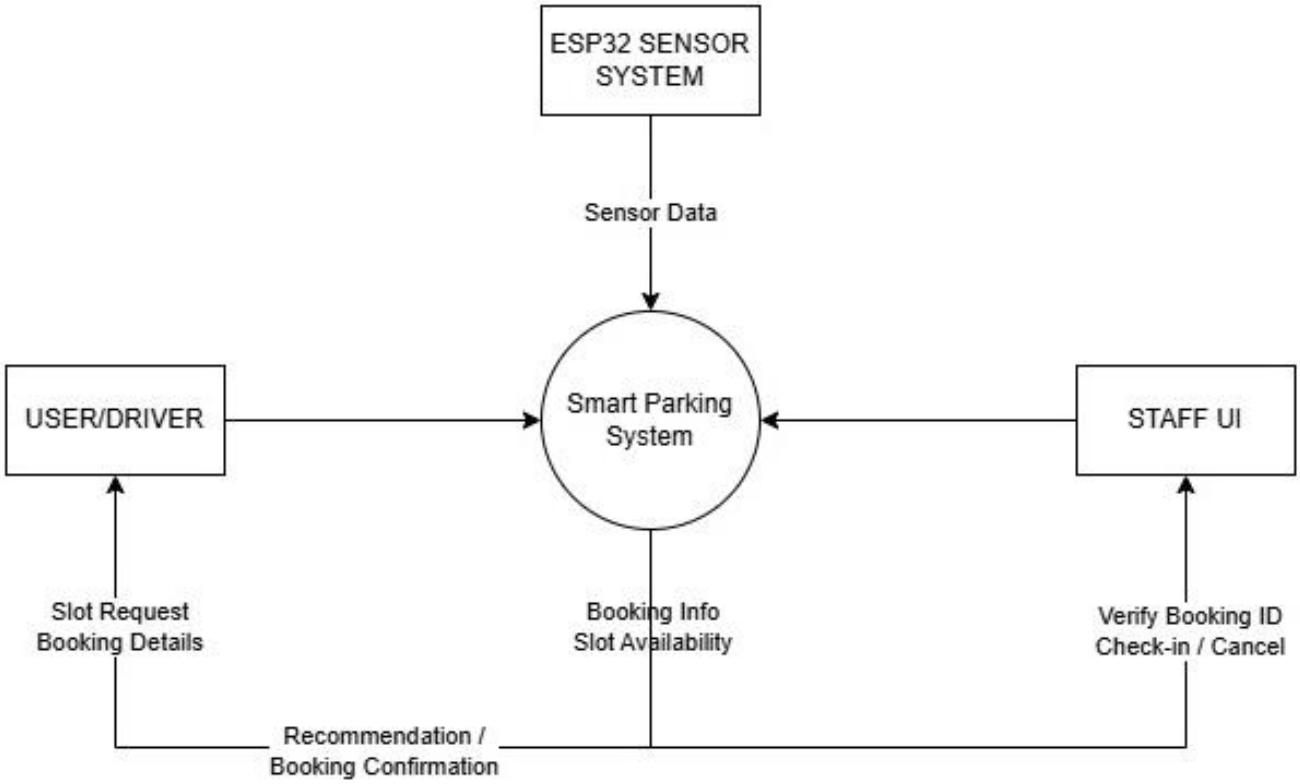


**The rules used in constructing data flow diagram are as follows:**

- Process should be named and numbered.
- The direction of flows is from top to bottom& from left to right.
- After exploding, lower-level details of process are to be numbered.
- The name of data stores, sources, destinations are written in upper cases processes & data flow name have first letter of each word capitalized.

# DATAFLOW DIAGRAM (DFD):

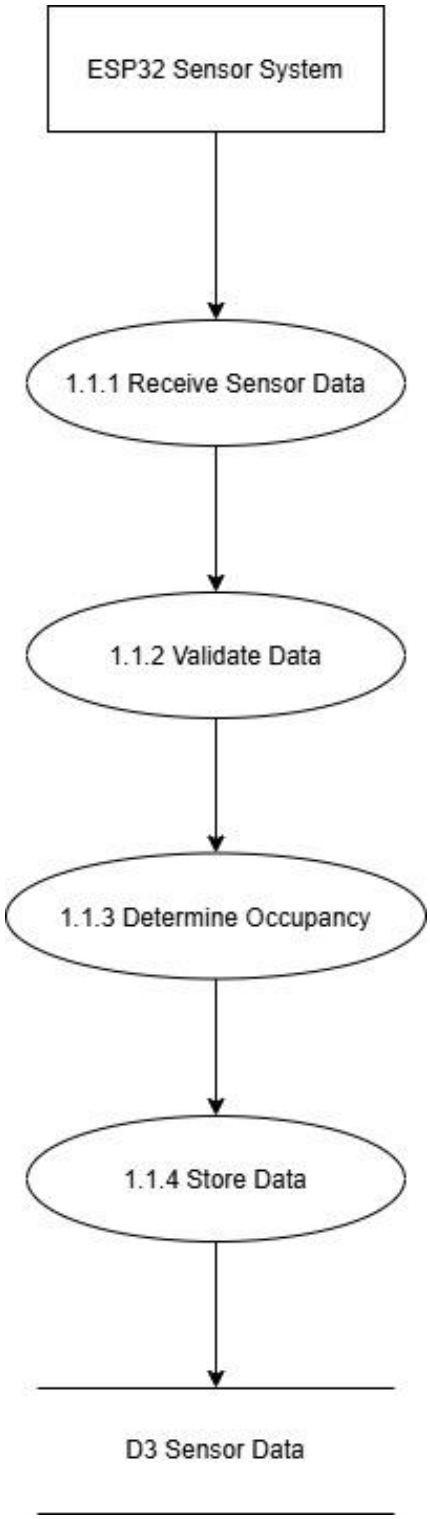
## 1.Level-0



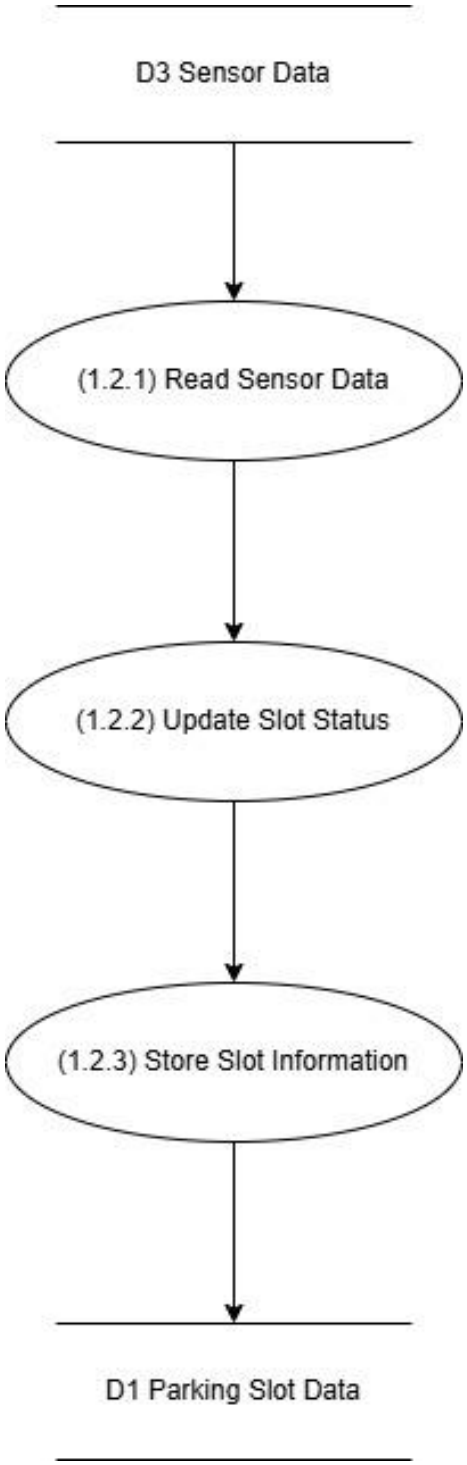
# 2.Level-1

ESP32 SENSOR SYSTEM
Sensor Readings
(1.1) Sensor Data Processing
Processed Sensor Data
Sensor Data
Sensor Status
(1.2) Parking Slot Management
Updated Slot Status
Parking Slot Data
Slot Availability
(1.3) AI Recomendation Processing
Recommended Slot
USER/DRIVER
Slot Info / Recomendation / Confirmation / Slot Details / Recommendation
Booking Request
(1.4) Booking Management
Booking Records
Booking Data
Updated Booking Status
Check-in / Cancelation Management
Verify Booking ID / Check-in / Cancel
STAFF UI

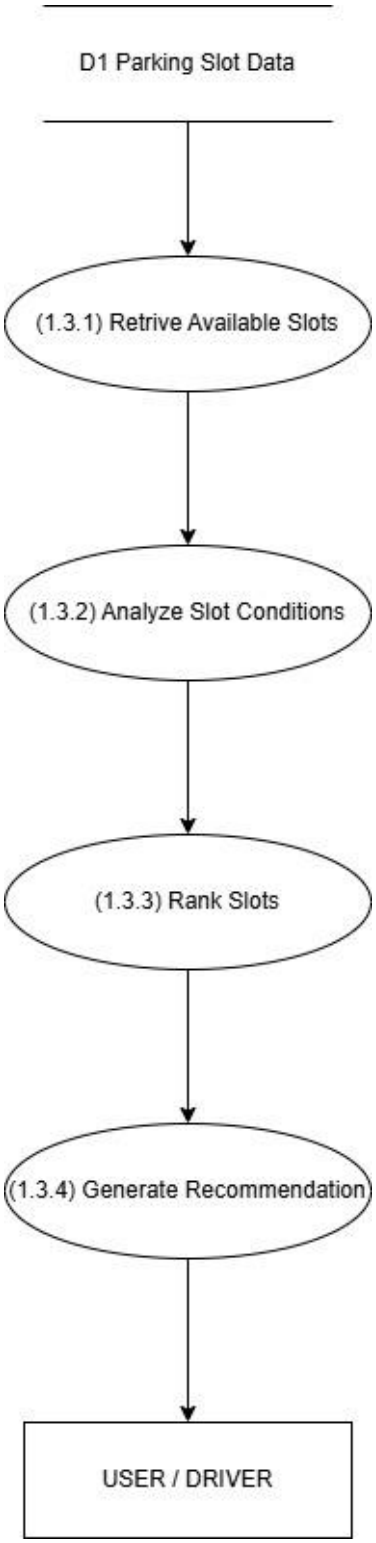
# 3.Level-1.1


ESP32 Sensor System
1.1.1 Receive Sensor Data
1.1.2 Validate Data
1.1.3 Determine Occupancy
1.1.4 Store Data
D3 Sensor Data

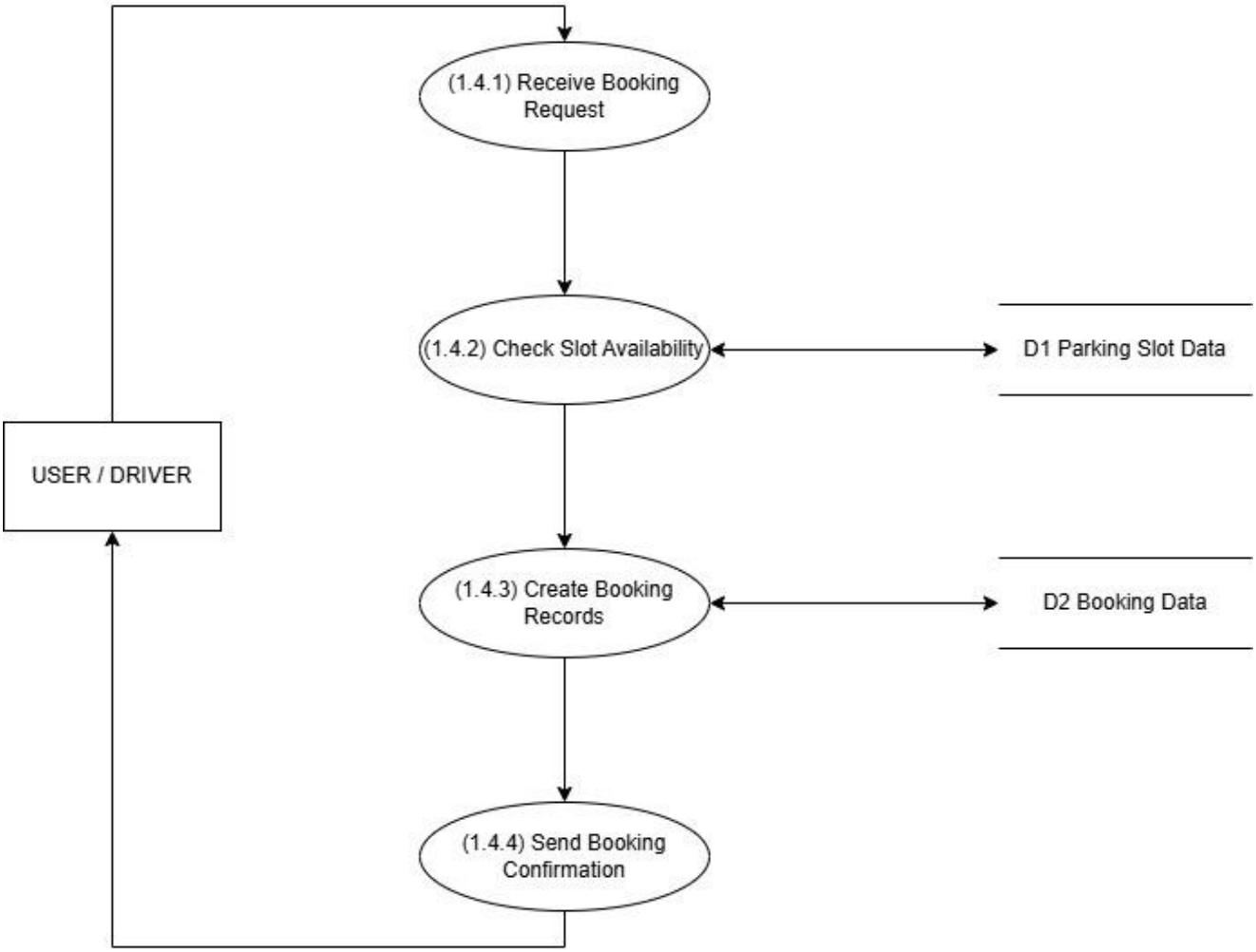
# 4.Level-1.2


D3 Sensor Data
(1.2.1) Read Sensor Data
(1.2.2) Update Slot Status
(1.2.3) Store Slot Information
D1 Parking Slot Data

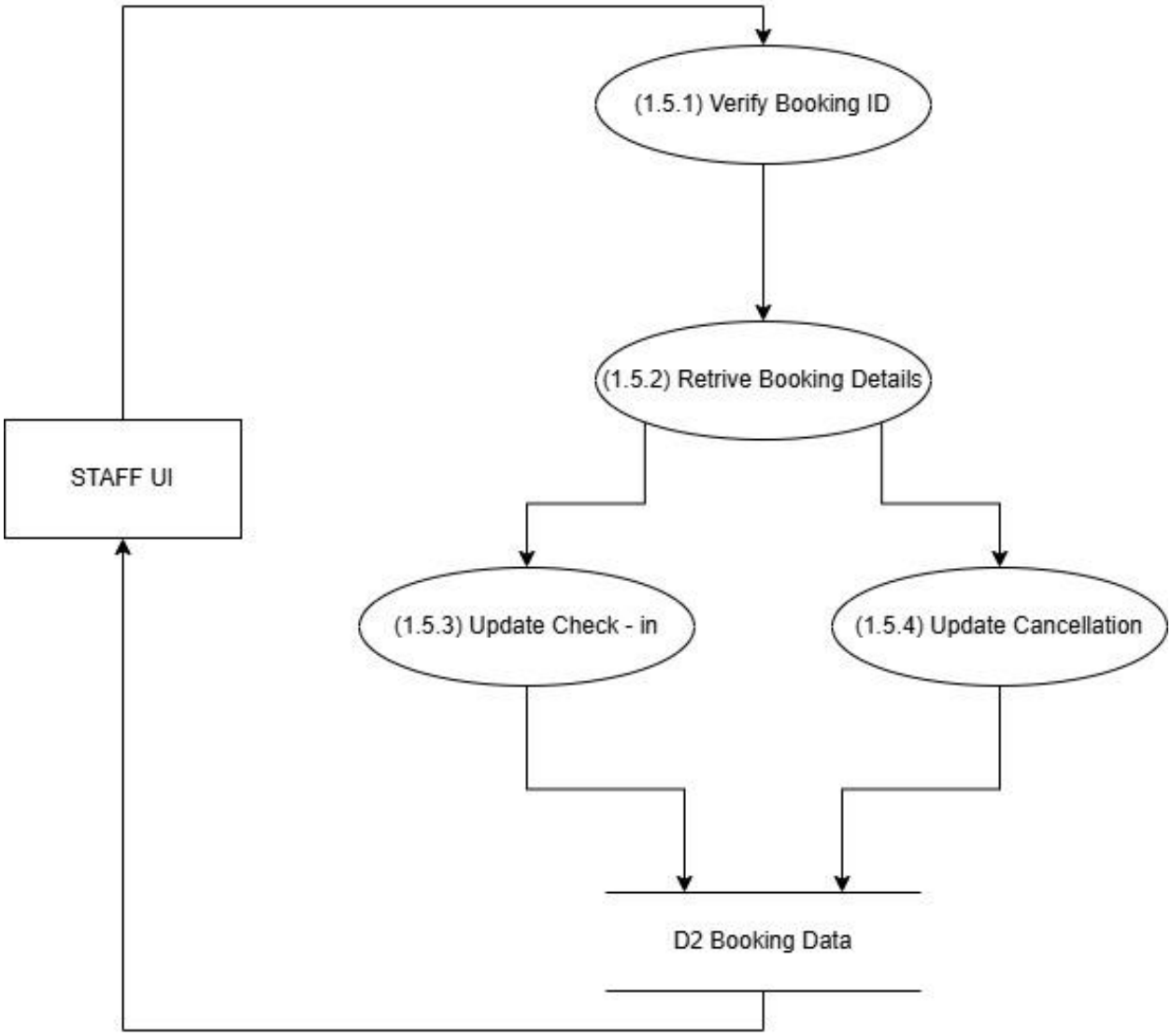
# 5.Level-1.3


D1 Parking Slot Data
(1.3.1) Retrive Available Slots
(1.3.2) Analyze Slot Conditions
(1.3.3) Rank Slots
(1.3.4) Generate Recommendation
USER / DRIVER

# 6.Level-1.4


(1.4.1) Receive Booking Request
(1.4.2) Check Slot Availability
D1 Parking Slot Data
USER / DRIVER
(1.4.3) Create Booking Records
D2 Booking Data
(1.4.4) Send Booking Confirmation

# 7.Level-1.5


(1.5.1) Verify Booking ID
(1.5.2) Retrive Booking Details
STAFF UI
(1.5.3) Update Check - in
(1.5.4) Update Cancellation
D2 Booking Data

# 5.4.DATABASE DESIGN

Database design is the process of producing a detailed data model of a database. This logical data model contains all the needed logical and physical design choices and physical storage parameters needed to generate a design in a data definition language, which can then be used to create a database. The term database design can be used to describe many different parts of the design of an overall database system. Principally, and most correctly, it can be thought of as the logical design of the base data structures used to store the data. In the relational model these are the tables and views. In an object database the entities and relationships map directly to object classes and named relationships. However, the term database design could also be used to apply to the overall process of designing, not just the base data structures, but also the forms and queries used as part of the overall database application within the database management system. The process of doing database design generally consists of a number of steps which will be carried out by the database designer.

Usually the designer must:

- Determine the relationships between the different data elements.
- Superimpose a logical structure upon the data on the basis of these relationships.

## Normalization

Normalization is the process of decomposing a set of relations with anomalies to produce smaller and well-structured relations that contain minimum redundancy. It is a formal process of deciding which attributes should be grouped together in a relation.

## First Normal Form (1NF)

First Normal form (1NF) is now considered to be part of the formal definition of relational model.1NF is designed to disallow multi valued attribute, composite attributes, and their combinations. It states that the domain of an attribute must include only atomic values. A domain is atomic, if elements of the domain are considered to be indivisible units. We say that a relational schema R is in 1NF if the domain of all attributes of 'R' is atomic.

## Second Normal Form (2NF)

Second Normal form (2NF) is based on the concept of functional dependency. A relation R is in 2NF if it is in 1NF and every non key attribute A of R is fully dependent on the primary key. That is, relation is said to be in 2NF if each attribute A in R meets one of the following criteria:

- It appears in the primary key.
- It is fully functionally dependent on the primary key.

## Third Normal Form (3NF)

Third Normal form (3NF) is based on the concept of transitive dependency. A relation is said to be in 3NF if it is in 2NF and has no transitive dependencies. That is all then on key attribute should be functionally determined by the primary key. In the proposed system all attributes of tables are fully depends on the primary key only that is all non key attributes are mutually independent.

## Database Design

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make

information access easy, quick, inexpensive and flexible for the user. In a database environment, common data are available in which several users can use. The concept behind a database is an integrated collection of data and provides a centralized access to the data from the program. When too many attributes are grouped together to form entities, some attributes are found to be entities themselves. Further normalization of these entities into attributes linked by common data elements to form relationships improves the effectiveness of the DBMS.

# 5.5.TABLE DESIGN

**Total no of tables : 6**

| Table Name | Description |
|---|---|
| **Parking_Slot** | Stores details of each parking slot including slot number, status (occupied/vacant), and last update time. |
| **Sensor** | Contains information about sensors installed in each parking slot, including sensor ID, type, and associated device and slot. |
| **Sensor_Reading** | Records real-time data captured by sensors such as reading value, occupancy status, and timestamp. |
| **ESP32_Device** | Maintains details of IoT devices (ESP32) including device ID, firmware version, status, and last seen time. |
| **Booking** | Stores user booking details such as booking ID, vehicle number, slot number, booking time, and status. |
| **AI_Recommendation_Log** | Stores AI-generated parking recommendations including recommended slot, request time, and score for optimization. |

## STRUCTURE OF THE TABLE

### 1.TABLE NAME:PARKING SLOT

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| slot_no | INT | PK | Unique slot number |
| slot_status | VARCHAR | | Occupied / Vacant |
| last_update_time | TIMESTAMP | | Last status update time |

### 2.TABLE NAME:SENSOR

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| sensor_id | INT | PK | Unique sensor ID |
| device_id | INT | FK | Linked ESP32 device |
| slot_no | INT | FK | Associated parking slot |
| pin_number | INT | | Hardware pin number |
| sensor_type | VARCHAR | | Type (IR / Ultrasonic) |

### 3.TABLE NAME:SENSOR READING

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| reading_id | INT | PK | Unique reading ID |
| sensor_id | INT | FK | Related sensor |
| slot_no | INT | FK | Related parking slot |

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| device_id | INT | FK | Source device |
| reading_value | FLOAT | | Sensor value |
| occupancy_status | VARCHAR | | Occupied / Vacant |
| reading_time | TIMESTAMP | | Time of reading |

## 4.TABLE NAME:ESP32 DEVICE

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| device_id | INT | PK | Unique device ID |
| device_name | VARCHAR | | Name of the device |
| firmware_version | VARCHAR | | Version of firmware |
| device_status | VARCHAR | | Active / Inactive |
| last_seen_time | TIMESTAMP | | Last communication time |

## 5.TABLE NAME:BOOKING

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| booking_id | INT | PK | Unique booking ID |
| slot_no | INT | FK | Booked parking slot |
| vehicle_no | VARCHAR | | Vehicle number |
| start_time | TIMESTAMP | | Booking start time |
| end_time | TIMESTAMP | | Booking end time |
| checkin_time | TIMESTAMP | | Actual check-in time |
| cancel_time | TIMESTAMP | | Cancellation time |
| booking_status | VARCHAR | | Active / Completed / Cancelled |
| created_time | TIMESTAMP | | Booking creation time |

## 6.TABLE NAME:AI RECOMENTATION LOG

| Column Name | Data Type | Key | Description |
|---|---|---|---|
| rec_id | INT | PK | Unique recommendation ID |
| slot_no | INT | FK | Recommended slot |
| request_time | TIMESTAMP | | Time of recommendation |
| score | FLOAT | | AI score for slot selection |

# 6. SYSTEM TESTING

# 6.1.SYSTEM TESTING

## TESTING

System testing is a comprehensive process consisting of multiple testing activities aimed at fully evaluating the functionality and performance of the developed smart parking system. It ensures that all components of the system, including IoT devices, cloud integration, user interface, and AI logic, are properly integrated and functioning as intended. Each test is designed with a specific objective, but collectively they verify that the system operates correctly as a complete unit.

Testing serves as the final stage of verification and validation within the development process. It is conducted to ensure that the system meets the specified requirements and performs reliably under different conditions. The primary goals of testing include affirming the quality of the product, identifying and eliminating residual errors from earlier development stages, validating the system as an effective solution to the defined problem, and ensuring that all required functionalities are correctly implemented.

In addition, testing helps in estimating the operational reliability and stability of the system. It ensures that real-time features such as parking slot updates, IoT sensor data transmission, and AI-based recommendations work accurately and without delay. Special attention is given to handling edge cases, such as incorrect sensor readings or network interruptions.

During the testing phase, major activities focus on examining system behavior, verifying outputs, and making necessary modifications to the source code. This iterative process helps in improving system performance and ensuring that the final product is efficient, reliable, and user-friendly.

## Type of Testing

Software testing determines the correctness, completeness, and quality of software being developed. Validation refers to the process of checking that the developed software meets the requirements specified by the user. The activities involved in the testing phase basically evaluate the capability of that system meets its requirements. The main objective of software testing is to detect errors in the

software. Errors occur if some part of the developed system is found to be incorrect, incomplete or inconsistent. Test techniques include, but are not limited to, the process of executing a program or application with the intent of finding software bugs (errors or other defects). It involves the execution of a software component or system to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test: Project Report 2025 SMART ASSIST 45 • Meets the requirements that guided its design and development.

• Responds correctly to all kinds of inputs.

• Performs its functions within an acceptable time.

• Is sufficiently usable.

• Can be installed and run in its intended environments.

• Achieves the general result its stakeholder's desire.

As the number of possible tests for even simple software components is practically infinite, all software testing uses some strategy to select tests that are feasible for the available time and resources. As a result, software testing typically (but not exclusively) attempts to execute a program or application with the intent of finding software bugs (errors or other defects). Software testing can provide objective, independent information about the quality of software and risk of its failure to users and/or sponsors. Software testing can be conducted as soon as executable software (even if partially complete) exists. The overall approach to software development often determines when and how testing is conducted. For example, in a phased process, most testing occurs after system requirements have been defined and then implemented in testable programs. In contrast, under an Agile approach, requirements, programming, and testing are often done concurrently.

**White Box Testing**

Tests are performed to ensure that all internal operations of the software are performed according to the specifications of the client. This is called White box testing. White box testing (also known as clear box testing, glass box testing, transparent box testing, and structural testing) is a method of testing software that tests internal structures or workings of an application, as opposed to its functionality (i.e., black-box testing). In white-box testing an internal perspective of the system, as well as programming skills, are used to design test cases. White-box testing can be applied at the unit, integration and system levels of the software testing process. Although traditional testers tended to think Project Report 2025 SMART ASSIST 46 of white-box testing as being done at the unit level, it is used for integration and system testing more frequently today. It can test paths within a unit, paths between units during integration, and between subsystems during a system–level test. Though this method of test design can uncover many errors or problems, it has the potential to miss unimplemented parts of the specification or missing requirements. The details entered by the administrator are saved and stored in the database, and testing is done to verify whether the control of each form or action is working in the exact way.

**Black Box Testing**

Tests are performed to ensure that each function is working properly. This is referred to as Black box testing. Black-box testing is a method of software testing that examines the functionality of an application (e.g., what the software does) without peering into its internal structures or workings. This method of test can be applied to virtually every level of software testing: unit, integration, system and acceptance. It typically comprises most if not all higher-level testing, but can also dominate unit testing as well. Test cases are built around specifications and requirements, i.e., what the application is supposed to do. Test cases are generally derived from external descriptions of the software, including specifications, requirements and design parameters. Although the tests used are primarily functional in nature, non-functional tests may also be used.

# 6.2.TESTING METHODOLOGIES

The following are the Testing Methodologies:

- Unit Testing
- Integration Testing
- Validation Testing
- Output Testing

**Unit Testing:**
Unit testing is performed to verify the functionality of individual modules of the system such as user interface, booking logic, IoT data handling, and AI recommendation. Each module is tested independently to ensure it works correctly without errors.

**Integration Testing:**
Integration testing is carried out to ensure that different modules of the system work together properly. It verifies the interaction between IoT devices (sensors and ESP32), Firebase cloud database, and the frontend interface to ensure smooth data flow and communication.

**Validation Testing:**
Validation testing is conducted to check whether the system meets the specified requirements and solves the intended problem. It ensures that all functionalities such as real-time slot updates, booking system, and AI recommendations are working as expected.

**Output Testing:**
Output testing focuses on verifying the correctness and accuracy of the system outputs. It ensures that results such as parking slot status, booking confirmations, AI recommendations, and payment (demo) messages are displayed correctly to the user.

# 6.3.TESTCASE TABLE

| Test Case ID | Test Description | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|---|
| TC_01 | Check system loading | Open web application | System loads successfully | As Expected | Pass |
| TC_02 | View parking availability | User opens dashboard | Available slots displayed correctly | As Expected | Pass |
| TC_03 | Sensor detects vehicle | Vehicle placed in slot | Slot status changes to “Occupied” | As Expected | Pass |
| TC_04 | Sensor detects empty slot | Vehicle removed | Slot status changes to “Vacant” | As Expected | Pass |
| TC_05 | AI recommendation | User requests slot | Best available slot suggested | As Expected | Pass |
| TC_06 | Booking a slot | Select available slot | Slot reserved and status updated | As Expected | Pass |
| TC_07 | Booking conflict | Select already occupied slot | Error message displayed | As Expected | Pass |
| TC_08 | Firebase real-time update | Change slot status | UI updates instantly | As Expected | Pass |
| TC_09 | Payment (Demo) process | Click pay | Payment success message displayed | As Expected | Pass |
| TC_10 | Invalid input handling | Enter incorrect data | Error/validation message shown | As Expected | Pass |
| TC_11 | Device communication | ESP32 sends data | Data updated in Firebase | As Expected | Pass |
| TC_12 | System performance | Multiple users access system | System responds without delay | As Expected | Pass |

# 7. IMPLEMENTATION

# 7.1.IMPLIMENTATION

The implementation phase of the **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT"** involves the development and integration of all system components, including the user interface, IoT hardware, cloud backend, and AI logic. The system is designed to operate in real time by continuously collecting data from parking slots and updating it on the cloud platform for user access.

The frontend of the system is developed using HTML, CSS, and JavaScript, providing a simple and user-friendly interface. It allows users to view parking slot availability, receive AI-based recommendations, book slots, and simulate payments. The interface dynamically updates parking information by fetching real-time data from the cloud database, ensuring that users always have access to the latest slot status.

The backend is implemented using Firebase, which serves as the cloud platform for data storage and synchronization. Firebase Realtime Database or Firestore is used to store information related to parking slots, sensor readings, bookings, and system logs. It enables real-time updates between the IoT devices and the user interface, ensuring seamless communication across all system components. Firebase also handles authentication and provides a scalable environment for the system.

The IoT module plays a crucial role in the implementation. Sensors such as ultrasonic or IR sensors are installed in each parking slot to detect the presence or absence of vehicles. These sensors are connected to a microcontroller (ESP32), which processes the sensor data and transmits it to Firebase via Wi-Fi. This ensures that any change in parking slot status is immediately reflected in the system.

The AI component is implemented using rule-based logic within the application. It analyzes real-time parking data and suggests the most suitable available slot to the user. This helps in reducing the time required to find parking and improves

overall efficiency. The recommendation logic can be further enhanced using machine learning techniques in future implementations.

The payment module is implemented as a demonstration feature. It simulates the process of digital payment without integrating an actual payment gateway. This allows users to understand the workflow of booking and payment within the system.

Overall, the implementation integrates multiple technologies to create a fully functional smart parking system. The combination of IoT for data collection, cloud computing for real-time synchronization, and AI for intelligent decision-making ensures that the system is efficient, scalable, and suitable for modern smart city applications.

# 8. SYSTEM MAINTENANCE

# 8.1.SYSTEM MAINTENANCE

System maintenance is an essential phase in the lifecycle of the **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT"**, ensuring that the system continues to operate efficiently, reliably, and securely after deployment. It involves monitoring, updating, and improving both software and hardware components to maintain optimal performance.

The maintenance process includes regular monitoring of the system to detect any issues related to IoT devices, cloud connectivity, or application performance. Sensor devices and the ESP32 microcontroller are periodically checked to ensure accurate data collection and transmission. Any hardware faults such as sensor malfunction or communication failure are identified and resolved promptly to avoid incorrect parking status updates.

On the software side, maintenance involves updating the frontend and backend components to improve functionality and fix bugs. Changes in Firebase configurations, database structure, or security rules are managed carefully to ensure continuous and secure data flow. Regular updates are also applied to enhance system performance, user interface design, and overall user experience.

The AI recommendation module is also maintained by refining its logic to improve accuracy in suggesting parking slots. As more data becomes available, the system can be adjusted to provide better recommendations and handle edge cases more effectively.

Database maintenance is performed to ensure data integrity and efficiency. This includes managing stored data, removing outdated records, and optimizing database structure for faster access and real-time updates. Backup mechanisms are also considered to prevent data loss and ensure system reliability.

In addition, preventive maintenance is carried out to reduce the chances of system failure. This includes routine checks, performance monitoring, and timely updates. Corrective maintenance is performed when errors or issues are detected, ensuring that the system is restored to normal operation quickly.

Overall, system maintenance ensures the long-term sustainability, reliability, and scalability of the smart parking system. It helps in adapting the system to future requirements and technological advancements while maintaining consistent performance.

# 9. FUTURE ENHANCEMENT

# 9.1.FUTURE ENHANCEMENT

The proposed **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT"** can be further improved and extended with several advanced features to enhance its functionality, scalability, and real-world applicability. Although the current system provides real-time parking updates, booking, and AI-based recommendations, there is significant scope for future development.

- One major enhancement is the integration of **advanced AI and machine learning algorithms**. Instead of rule-based logic, predictive models can be implemented to forecast parking availability based on historical data, time patterns, and user behavior. This would enable the system to provide more accurate and intelligent recommendations.
- Another important improvement is the inclusion of a **mobile application (Android/iOS)**. A dedicated mobile app would provide better accessibility, push notifications, GPS-based navigation to parking slots, and a more user-friendly experience compared to a web-based interface.
- The system can also be enhanced by integrating a **real-time payment gateway** such as UPI, credit/debit cards, or digital wallets. This would replace the current demo payment module and enable secure, real-world transactions within the system.
- To improve automation and accuracy, **camera-based vehicle detection** using computer vision can be implemented. This would complement or replace sensor-based detection and provide features such as number plate recognition and security monitoring.
- Another enhancement is the implementation of **dynamic pricing mechanisms**, where parking charges vary based on demand, time, or location. This can help in better space utilization and revenue optimization.
- The system can be expanded to support **multi-location parking management**, allowing integration of multiple parking areas into a single platform. This would make the system suitable for large-scale deployment in smart cities.

- Additionally, incorporating **notification and alert systems** (SMS, email, or app notifications) can improve user experience by informing users about booking confirmations, slot availability, and reminders.
- Security can be enhanced by implementing **strong authentication mechanisms** and data encryption to protect user and system data. Regular updates and monitoring can further improve system reliability.
- Overall, these future enhancements can transform the current prototype into a fully functional, scalable, and intelligent smart parking solution suitable for real-world deployment and smart city infrastructure.

# 10. CONCLUSION

# 10.1.CONCLUSION

The **"Real-Time Smart Parking System Using AI, Cloud Computing, and IoT"** successfully demonstrates an efficient and modern solution to the growing problem of parking management. The system integrates IoT sensors, cloud-based data handling, and AI-driven recommendations to provide real-time information about parking slot availability, thereby reducing the time and effort required for users to find parking.

Through the implementation of this system, users are able to view live parking status, receive intelligent slot suggestions, and book parking spaces through a simple and user-friendly interface. The use of Firebase ensures real-time synchronization between hardware devices and the application, while the IoT components enable accurate detection of vehicle presence. The AI module further enhances system efficiency by guiding users to the most suitable available slots.

The project also demonstrates the effective integration of multiple technologies, including web development, cloud computing, and embedded systems, into a single unified platform. Despite being a prototype with a demo-based payment module, the system lays a strong foundation for real-world applications in smart cities and automated parking management systems.

In conclusion, the developed system is reliable, scalable, and capable of improving parking efficiency in urban environments. With further enhancements such as real payment integration, mobile applications, and advanced AI techniques, the system can be extended into a fully functional smart parking solution suitable for large-scale deployment.

# 11. BIBLIOGRAPHY

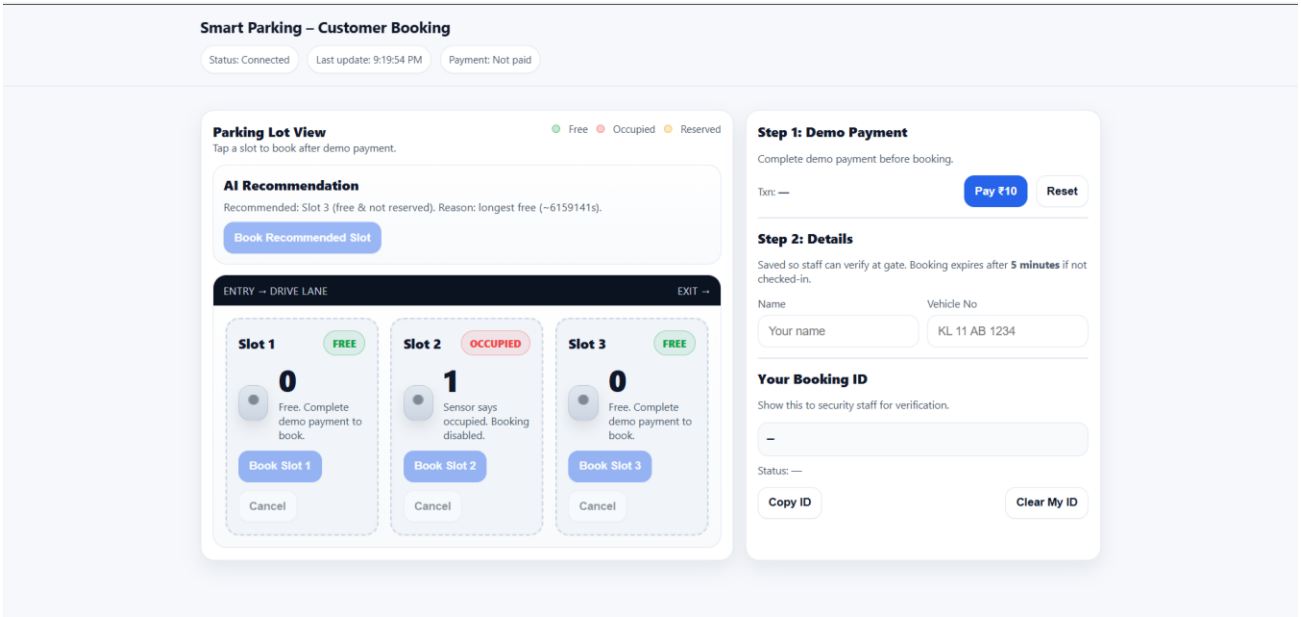
# 11.1.BIBLIOGRAPHY


- **Ian Sommerville Software Engineering, *Software Engineering*, 10th Edition, Pearson Education.**
- **Raj Kamal Embedded Systems, *Embedded Systems: Architecture, Programming and Design*, McGraw Hill Education.**
- **Google Firebase, *Firebase Documentation*. Available: https://firebase.google.com/docs**
- **Arduino, *Arduino Official Documentation*. Available: https://www.arduino.cc**
- **ESP32 Espressif, *ESP32 Technical Reference Manual*.**
- **Ethan Brown Learning JavaScript, *Learning JavaScript*, O'Reilly Media.**
- **Jon Duckett HTML and CSS, *HTML and CSS: Design and Build Websites*, Wiley.**
- **IEEE, *Research Papers on Smart Parking Systems*. Available: https://ieeexplore.ieee.org**
- **W3Schools, *Web Development Tutorials*. Available: https://www.w3schools.com**
- **GeeksforGeeks, *Programming and Development Resources*. Available: https://www.geeksforgeeks.org**

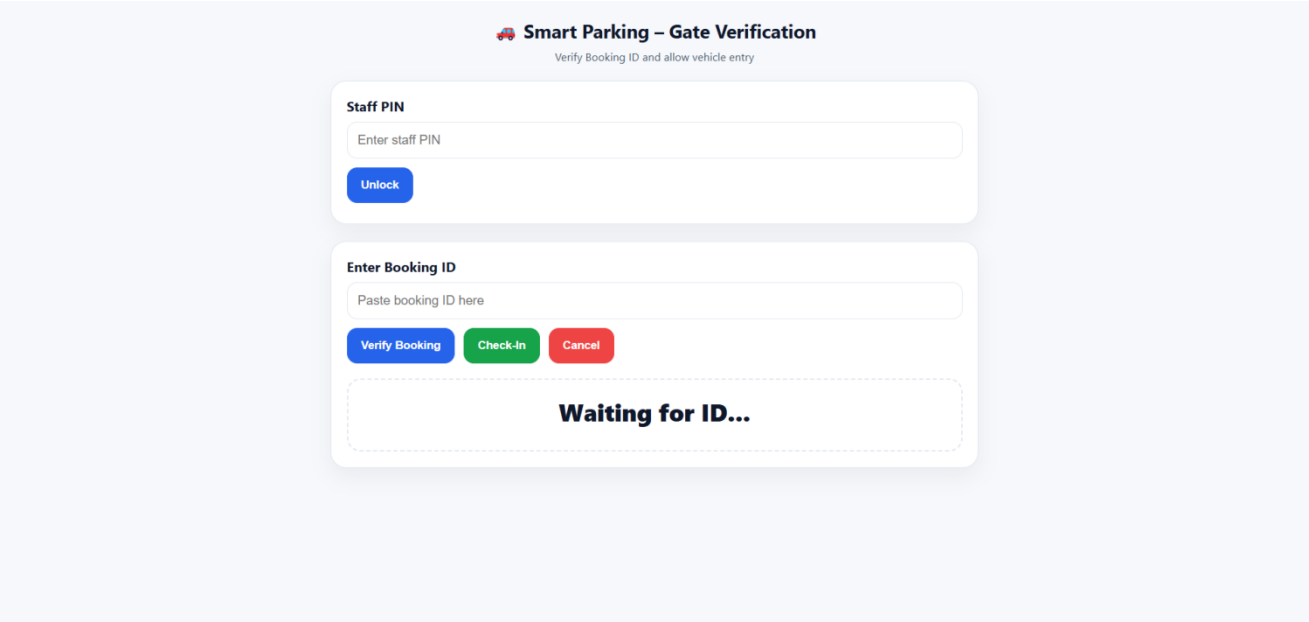
# 12. APPENDIX

# 12.1.USER INTERFACE

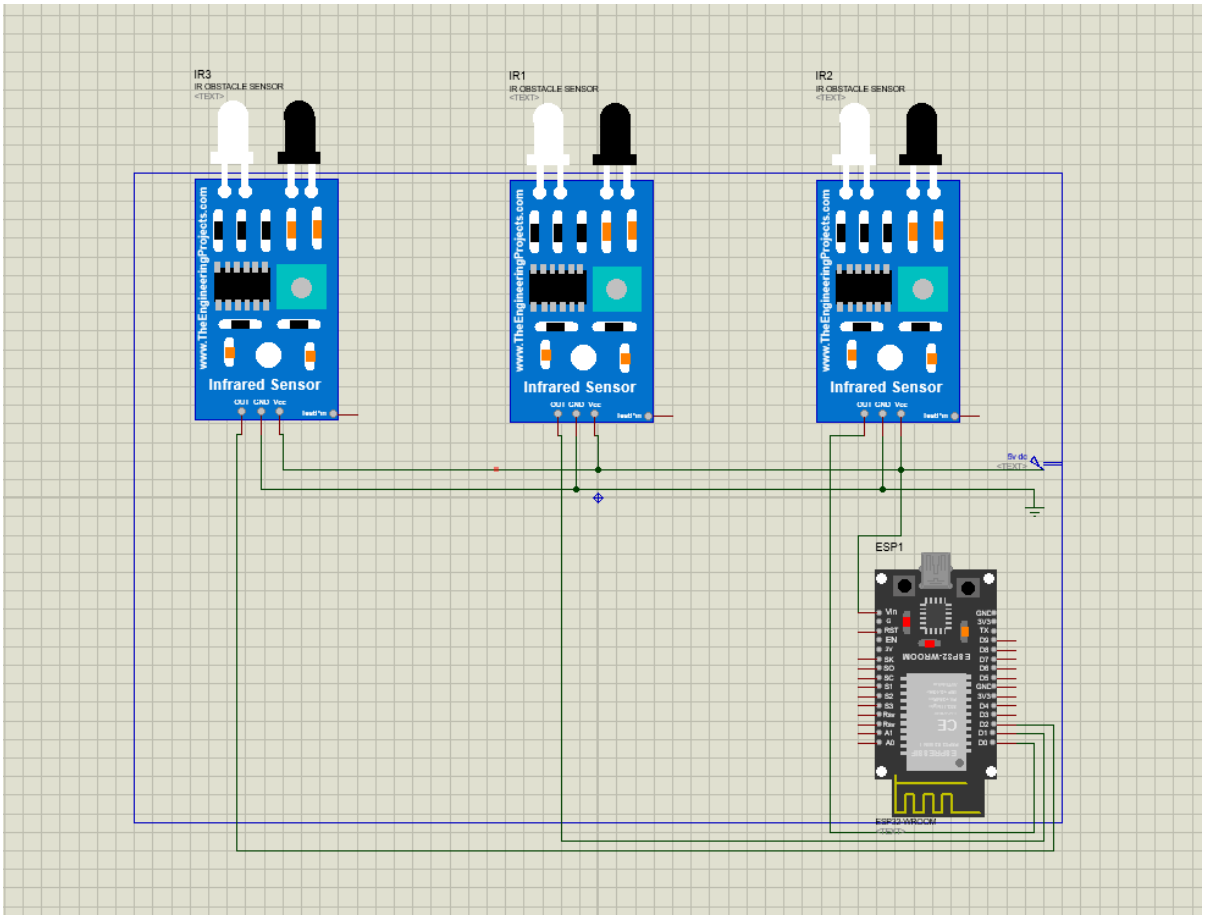
## USER/DRIVER INTERFACE:

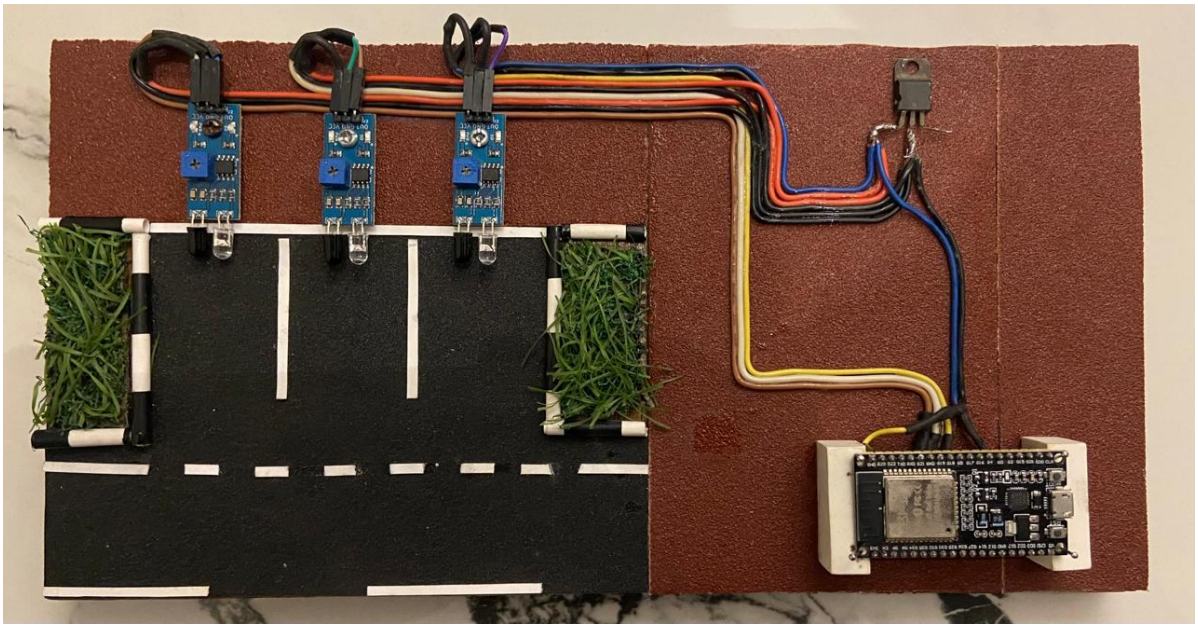



## STAFF'S INTERFACE:



# 12.2.HARDWARE SETUP IMAGES



# 12.3.SOURCE CODE

**CUSTOMER INTERFACE CODE:**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Smart Parking – Customer Booking</title>

  <style>
    :root{
      --bg:#f6f8fc;
      --card:#ffffff;
      --text:#0f172a;
      --muted:#475569;
      --border:#e2e8f0;
      --shadow: 0 10px 24px rgba(15,23,42,.08);
      --blue:#2563eb;
      --green:#16a34a;
      --red:#ef4444;
      --amber:#f59e0b;
      --slot:#f1f5f9;
```

```
  --lane:#0b1220;
  --laneText:#cbd5e1;
}

*{box-sizing:border-box}
body{margin:0;font-family:system-ui,-apple-system,Segoe UI,Roboto,Arial;background:var(--bg);color:var(--text)}
header{position:sticky;top:0;z-index:10;background:rgba(246,248,252,.85);backdrop-filter:blur(10px);border-bottom:1px solid var(--border)}
.wrap{max-width:1100px;margin:0 auto;padding:14px 16px}
h1{margin:0;font-size:18px}
.sub{margin-top:6px;color:var(--muted);font-size:12.5px}
.row{display:flex;gap:10px;flex-wrap:wrap;align-items:center}
.pill{padding:8px 10px;border:1px solid var(--border);border-radius:999px;background:#fff;color:var(--muted);font-size:12px;box-shadow:0 1px 0 rgba(15,23,42,.03)}
.grid{display:grid;grid-template-columns: 1.2fr .8fr;gap:14px;margin-top:14px}
@media (max-width:980px){.grid{grid-template-columns:1fr}}

.card{
  background:var(--card);
  border:1px solid var(--border);
  border-radius:16px;
  box-shadow: var(--shadow);
  padding:14px;
}
```

```
/* Parking lot */
.lotHeader{display:flex;justify-content:space-between;align-items:flex-start;gap:12px;flex-wrap:wrap}
.lotTitle{font-weight:800}
.lotLegend{display:flex;gap:10px;flex-wrap:wrap;color:var(--muted);font-size:12px}
.dot{width:10px;height:10px;border-radius:50%}
.dFree{background:rgba(22,163,74,.25);border:1px solid rgba(22,163,74,.55)}
.dOcc{background:rgba(239,68,68,.25);border:1px solid rgba(239,68,68,.55)}
.dRes{background:rgba(245,158,11,.25);border:1px solid rgba(245,158,11,.55)}

.lot{
  margin-top:12px;
  border-radius:16px;
  overflow:hidden;
  border:1px solid var(--border);
  background: linear-gradient(180deg, #ffffff, #f8fafc);
}

.lane{
  background: var(--lane);
  color: var(--laneText);
  padding:10px 12px;
  display:flex;
```

```
  justify-content:space-between;
  align-items:center;
  font-size:12px;
  letter-spacing:.4px;
}

.slots{
  padding:14px;
  display:grid;
  grid-template-columns:repeat(3,1fr);
  gap:14px;
}
@media (max-width:720px){.slots{grid-template-columns:1fr}}

.slot{
  position:relative;
  background:var(--slot);
  border:2px dashed #cbd5e1;
  border-radius:16px;
  padding:14px;
  min-height:210px;
  display:flex;
  flex-direction:column;
  justify-content:space-between;
```

```
  box-shadow: inset 0 0 0 1px rgba(15,23,42,.02);
}

.slotTop{display:flex;justify-content:space-between;align-items:center}
.slotName{font-weight:800}

.badge{
  padding:6px 10px;
  border-radius:999px;
  font-size:12px;
  font-weight:800;
  border:1px solid var(--border);
  background:#fff;
  color:var(--muted);
}
.bFree{border-color:rgba(22,163,74,.35);background:rgba(22,163,74,.10);color:var(--green)}
.bOcc{border-color:rgba(239,68,68,.35);background:rgba(239,68,68,.10);color:var(--red)}
.bRes{border-color:rgba(245,158,11,.35);background:rgba(245,158,11,.12);color:#b45309}

.slotMid{display:flex;gap:12px;align-items:center;margin-top:10px}
.car{
  width:74px;height:42px;border-radius:14px;
  background: linear-gradient(180deg,#e2e8f0,#cbd5e1);
  border:1px solid rgba(15,23,42,.10);
```

```
  position:relative;
  box-shadow: 0 8px 16px rgba(15,23,42,.10);
}
.car:before,.car:after{
  content:"";position:absolute;top:10px;width:12px;height:12px;border-radius:50%;
  background:#0f172a;opacity:.25
}
.car:before{left:10px}
.car:after{right:10px}
.value{
  font-size:36px;font-weight:900;line-height:1;
}
.hint{color:var(--muted);font-size:12.5px;margin-top:6px}

.btnRow{display:flex;gap:10px;flex-wrap:wrap;margin-top:10px}
button{
  padding:10px 12px;border-radius:12px;border:1px solid var(--border);
  background:#fff;color:var(--text);cursor:pointer;font-weight:700;
  box-shadow:0 1px 0 rgba(15,23,42,.03);
}
button.primary{background:var(--blue);border-color:rgba(37,99,235,.35);color:#fff}
button.secondary{background:#fff;opacity:.9}
button:disabled{opacity:.45;cursor:not-allowed}
```

```
/* Right panel */
.sectionTitle{font-weight:900;margin:0 0 6px}
.small{color:var(--muted);font-size:12.5px}
input{
  width:100%;
  background:#fff;border:1px solid var(--border);
  border-radius:12px;padding:10px 12px;outline:none;
  font-size:14px;
}
.fieldGrid{display:grid;grid-template-columns:1fr 1fr;gap:10px;margin-top:10px}
@media (max-width:520px){.fieldGrid{grid-template-columns:1fr}}

.divider{height:1px;background:var(--border);margin:12px 0}

.codebox{
  font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono",
monospace;
  background:#f8fafc;border:1px solid var(--border);
  padding:10px;border-radius:12px;word-break:break-all;
  color:#0f172a;
}

.aiBox{
  border:1px solid var(--border);
```

```
    background: linear-gradient(180deg,#ffffff,#f8fafc);
    border-radius:16px;
    padding:12px;
    margin-top:12px;
  }
  .aiRow{display:flex;justify-content:space-between;gap:10px;align-items:flex-start;flex-wrap:wrap}
  .aiBig{font-weight:900}
  .aiText{color:var(--muted);font-size:13px;margin-top:6px}
 </style>
</head>

<body>
<header>
 <div class="wrap">
  <h1>Smart Parking – Customer Booking</h1>
  <div class="sub">
   <!--Live: <code>/parking</code> · History: <code>/bookings/&lt;id&gt;</code> · Lock: <code>/activeBookings</code> · Meta: <code>/meta</code>-->
  </div>
  <div class="row" style="margin-top:10px">
   <div class="pill" id="conn">Status: Connecting…</div>
   <div class="pill" id="last">Last update: —</div>
   <div class="pill" id="payState">Payment: Not paid</div>
  </div>
```

```
  </div>
</header>

<main class="wrap">
  <div class="grid">

    <!-- LEFT: Parking lot -->
    <section class="card">
      <div class="lotHeader">
        <div>
          <div class="lotTitle">Parking Lot View</div>
          <div class="small">Tap a slot to book after demo payment.</div>
        </div>
        <div class="lotLegend">
          <span class="row"><span class="dot dFree"></span> Free</span>
          <span class="row"><span class="dot dOcc"></span> Occupied</span>
          <span class="row"><span class="dot dRes"></span> Reserved</span>
        </div>
      </div>

      <div class="aiBox">
        <div class="aiRow">
          <div>
            <div class="aiBig">AI Recommendation</div>
```

```
          <div class="aiText" id="aiText">Waiting for live data…</div>
        </div>
        <button id="bookRecommended" class="primary" disabled>Book Recommended
Slot</button>
      </div>
    </div>

    <div class="lot">
      <div class="lane">
        <span>ENTRY → DRIVE LANE</span>
        <span>EXIT →</span>
      </div>

      <div class="slots">

        <!-- SLOT 1 -->
        <div class="slot" id="card1">
          <div>
            <div class="slotTop">
              <div class="slotName">Slot 1</div>
              <span class="badge" id="b1">—</span>
            </div>

            <div class="slotMid">
```

```
      <div class="car" aria-hidden="true"></div>
      <div>
        <div class="value" id="v1">—</div>
        <div class="hint" id="h1">0 = Free, 1 = Occupied</div>
      </div>
    </div>
  </div>

  <div class="btnRow">
    <button id="book1" class="primary">Book Slot 1</button>
    <button id="cancel1" class="secondary">Cancel</button>
  </div>
</div>

<!-- SLOT 2 -->
<div class="slot" id="card2">
  <div>
    <div class="slotTop">
      <div class="slotName">Slot 2</div>
      <span class="badge" id="b2">—</span>
    </div>

    <div class="slotMid">
      <div class="car" aria-hidden="true"></div>
```

```
      <div>
        <div class="value" id="v2">—</div>
        <div class="hint" id="h2">0 = Free, 1 = Occupied</div>
      </div>
    </div>
  </div>

  <div class="btnRow">
    <button id="book2" class="primary">Book Slot 2</button>
    <button id="cancel2" class="secondary">Cancel</button>
  </div>
</div>

<!-- SLOT 3 -->
<div class="slot" id="card3">
  <div>
    <div class="slotTop">
      <div class="slotName">Slot 3</div>
      <span class="badge" id="b3">—</span>
    </div>

    <div class="slotMid">
      <div class="car" aria-hidden="true"></div>
      <div>
```

```
            <div class="value" id="v3">—</div>
             <div class="hint" id="h3">0 = Free, 1 = Occupied</div>
           </div>
          </div>
         </div>

         <div class="btnRow">
           <button id="book3" class="primary">Book Slot 3</button>
           <button id="cancel3" class="secondary">Cancel</button>
         </div>
        </div>

      </div>
    </div>
  </section>

  <!-- RIGHT: Payment + User details + Booking ID -->
  <aside class="card">
    <p class="sectionTitle">Step 1: Demo Payment</p>
    <p class="small">Complete demo payment before booking.</p>
    <div class="row" style="justify-content:space-between;margin-top:10px">
      <div class="small">Txn: <b id="txnText">—</b></div>
      <div class="row">
        <button id="payBtn" class="primary">Pay ₹10</button>
```

```
      <button id="resetPayBtn">Reset</button>
    </div>
  </div>

  <div class="divider"></div>

  <p class="sectionTitle">Step 2: Details</p>
  <p class="small">Saved so staff can verify at gate. Booking expires after <b>5 minutes</b> if not checked-in.</p>

  <div class="fieldGrid">
    <div>
      <div class="small" style="margin-bottom:6px">Name</div>
      <input id="name" placeholder="Your name" value="ABC" />
    </div>
    <div>
      <div class="small" style="margin-bottom:6px">Vehicle No</div>
      <input id="vehicle" placeholder="KL 11 AB 1234" value="111" />
    </div>
  </div>

  <div class="divider"></div>

  <p class="sectionTitle">Your Booking ID</p>
```

```
      <p class="small">Show this to security staff for verification.</p>


      <div class="codebox" id="myIdBox" style="margin-top:10px">—</div>
      <div class="small" id="myIdStatus" style="margin-top:8px">Status: —</div>


      <div class="row" style="margin-top:12px;justify-content:space-between">
        <button id="copyIdBtn">Copy ID</button>
        <button id="clearLocalBtn">Clear My ID</button>
      </div>
    </aside>


  </div>
</main>


<script type="module">
  import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.5/firebase-app.js";
  import {
    getDatabase, ref, onValue, push, set, update, remove, get
  } from "https://www.gstatic.com/firebasejs/10.12.5/firebase-database.js";


  // ⚙ Your Firebase web config
  const firebaseConfig = {
    apiKey: "AIzaSyDTAas1droAkeShhrnvoJhDCc74uJQQhEA",
    authDomain: "esp32-parking-8feef.firebaseapp.com",
```

```
  databaseURL: "https://esp32-parking-8feef-default-rtdb.firebaseio.com",
  projectId: "esp32-parking-8feef",
  storageBucket: "esp32-parking-8feef.appspot.com",
  messagingSenderId: "89076263554",
  appId: "1:89076263554:web:61669b435d5c272307e29b",
};

const app = initializeApp(firebaseConfig);
const db = getDatabase(app);

const el = (id) => document.getElementById(id);
const nowMs = () => Date.now();

// ---- local booking identity (demo) ----
const LS_KEY = "SMART_PARKING_BOOKING_ID";
function getMyBookingId(){ return localStorage.getItem(LS_KEY) || ""; }
function setMyBookingId(id){ localStorage.setItem(LS_KEY, id); renderMyId(); }
function clearMyBookingId(){ localStorage.removeItem(LS_KEY); renderMyId(); }

function renderMyId() {
  const id = getMyBookingId();
  el("myIdBox").textContent = id ? id : "—";
}
```

```
// ---- demo payment ----
let paid = false;
let txn = "";

function setPaidState(v) {
  paid = v;
  el("payState").textContent = paid ? "Payment: Paid ₹10" : "Payment: Not paid";
  el("txnText").textContent = txn || "—";
  renderAll();
  computeRecommendation();
}

el("payBtn").onclick = () => {
  txn = "DEMO-" + Math.random().toString(36).slice(2, 8).toUpperCase();
  setPaidState(true);
  alert("Demo payment success ");
};

el("resetPayBtn").onclick = () => {
  paid = false;
  txn = "";
  setPaidState(false);
  alert("Payment reset.");
};
```

```
// ---- live state ----
let parking = { slot1: 0, slot2: 0, slot3: 0 };
let active = { slot1: null, slot2: null, slot3: null };
let lastChange = { slot1: 0, slot2: 0, slot3: 0 };

let recommendedSlot = null;

function paintBadge(n, type, text){
  const b = el("b"+n);
  b.textContent = text;
  b.className = "badge " + (type === "free" ? "bFree" : type === "occ" ? "bOcc" : "bRes");
}

function setSlotUI(n) {
  const key = "slot" + n;
  const occ = Number(parking[key] ?? 0);
  const lock = active[key]; // booking id or null

  el("v"+n).textContent = occ;
  const hint = el("h"+n);

  if (occ === 1) {
    paintBadge(n, "occ", "OCCUPIED");
```

```
      hint.textContent = "Sensor says occupied. Booking disabled.";
    } else if (lock) {
      paintBadge(n, "res", "RESERVED");
      hint.textContent = "Reserved (booking ID exists).";
    } else {
      paintBadge(n, "free", "FREE");
      hint.textContent = paid ? "Free. You can book now." : "Free. Complete demo payment to
book.";
    }

    const canBook = paid && occ === 0 && !lock;
    el("book"+n).disabled = !canBook;

    const myId = getMyBookingId();
    const canCancel = !!myId && lock === myId;
    el("cancel"+n).disabled = !canCancel;
  }

  function renderAll() { [1,2,3].forEach(setSlotUI); }

  // ---- IMPORTANT FIX: if my booking was cancelled/checked-in, clear localStorage ----
  async function refreshMyBookingStatus() {
    const id = getMyBookingId();
    if (!id) {
```

```
    el("myIdStatus").textContent = "Status: —";
    return;
  }
  const snap = await get(ref(db, `bookings/${id}`));
  if (!snap.exists()) {
    el("myIdStatus").textContent = "Status: Not found (cleared)";
    clearMyBookingId();
    return;
  }
  const status = String(snap.val().status || "");
  el("myIdStatus").textContent = "Status: " + status;

  if (status !== "reserved") {
    clearMyBookingId();
  }
}

// ---- booking actions ----
async function bookSlot(n) {
  if (!paid) return alert("Complete demo payment first.");

  const name = el("name").value.trim();
  const vehicle = el("vehicle").value.trim();
  if (!name || !vehicle) return alert("Enter Name and Vehicle No.");
```

```
  const key = "slot" + n;
  if (Number(parking[key] ?? 0) === 1) return alert("Slot is occupied. Cannot book.");
  if (active[key]) return alert("Slot already reserved.");

  await refreshMyBookingStatus();
  if (getMyBookingId()) return alert("You already have an active booking ID. Cancel/check-in
first.");

  const expiresAt = nowMs() + 5 * 60 * 1000;

  const bookingRef = push(ref(db, "bookings"));
  const bookingId = bookingRef.key;

  await set(bookingRef, {
    slot: n,
    status: "reserved",
    paid: true,
    amount: 10,
    txn,
    customerName: name,
    vehicleNo: vehicle,
    createdAt: nowMs(),
    expiresAt
```

```
  });

  await set(ref(db, `activeBookings/slot${n}`), bookingId);

  setMyBookingId(bookingId);
  alert("Booked  Booking ID created.");
  await refreshMyBookingStatus();
}

async function cancelMyBooking(n) {
  const myId = getMyBookingId();
  if (!myId) return alert("No active booking ID found.");

  const lockSnap = await get(ref(db, `activeBookings/slot${n}`));
  if (!lockSnap.exists() || lockSnap.val() !== myId) {
    alert("This slot is not reserved by your ID.");
    await refreshMyBookingStatus();
    return;
  }

  await update(ref(db, `bookings/${myId}`), { status: "cancelled", cancelledAt: nowMs() });
  await remove(ref(db, `activeBookings/slot${n}`));

  alert("Booking cancelled.");
```

```
  clearMyBookingId();
  await refreshMyBookingStatus();
}

[1,2,3].forEach(n => {
  el("book"+n).onclick = () => bookSlot(n);
  el("cancel"+n).onclick = () => cancelMyBooking(n);
});

el("copyIdBtn").onclick = async () => {
  const id = getMyBookingId();
  if (!id) return alert("No booking ID.");
  await navigator.clipboard.writeText(id);
  alert("Booking ID copied.");
};

el("clearLocalBtn").onclick = () => {
  clearMyBookingId();
  alert("Cleared your local booking ID.");
};

// ---- AI recommendation (longest-free using meta timestamps) ----
function computeRecommendation() {
  const candidates = [1,2,3].filter(n => {
```

```
    const key = "slot"+n;
    const occ = Number(parking[key] ?? 0) === 1;
    const res = !!active[key];
    return !occ && !res;
  });

  if (!candidates.length) {
    recommendedSlot = null;
    el("aiText").textContent = "No free slot available (or all free slots are reserved).";
    el("bookRecommended").disabled = true;
    return;
  }

  candidates.sort((a,b) => (lastChange["slot"+a] || 0) - (lastChange["slot"+b] || 0));
  recommendedSlot = candidates[0];

  const ageSec = lastChange["slot"+recommendedSlot]
    ? Math.max(0, Math.floor((nowMs() - lastChange["slot"+recommendedSlot]) / 1000))
    : 0;

  el("aiText").textContent =
    `Recommended: Slot ${recommendedSlot} (free & not reserved). ` +
    (ageSec ? `Reason: longest free (~${ageSec}s).` : "");
```

```
  el("bookRecommended").disabled = !(paid); // also must be paid
}

el("bookRecommended").onclick = () => {
  if (!recommendedSlot) return;
  bookSlot(recommendedSlot);
};

// ---- expiry sweeper (client-side demo) ----
async function expireIfNeeded() {
  const snap = await get(ref(db, "activeBookings"));
  const locks = snap.val() || {};
  for (const k of ["slot1","slot2","slot3"]) {
    const bookingId = locks[k];
    if (!bookingId) continue;

    const bSnap = await get(ref(db, `bookings/${bookingId}`));
    if (!bSnap.exists()) {
      await remove(ref(db, `activeBookings/${k}`));
      continue;
    }

    const b = bSnap.val();
    const status = String(b.status || "");
```

```
    const expiresAt = Number(b.expiresAt || 0);

    if (status === "reserved" && expiresAt && nowMs() > expiresAt) {
      await update(ref(db, `bookings/${bookingId}`), { status: "expired", expiredAt: nowMs() });
      await remove(ref(db, `activeBookings/${k}`));
    }
  }
}

// ---- listeners ----
onValue(ref(db, "parking"), (snap) => {
  el("conn").textContent = "Status: Connected";
  parking = snap.val() || { slot1:0, slot2:0, slot3:0 };
  renderAll();
  el("last").textContent = "Last update: " + new Date().toLocaleTimeString();
  computeRecommendation();
}, (err) => {
  el("conn").textContent = "Status: Error - " + err.code;
  console.error(err);
});

onValue(ref(db, "activeBookings"), (snap) => {
  active = snap.val() || { slot1:null, slot2:null, slot3:null };
  renderAll();
```

```
    computeRecommendation();
  });

  onValue(ref(db, "meta"), (snap) => {
    const meta = snap.val() || {};
    [1,2,3].forEach(n => {
      const key = "slot"+n;
      lastChange[key] = Number(meta[key]?.lastChange ?? 0);
    });
    computeRecommendation();
  });

  // initial
  renderMyId();
  setPaidState(false);
  refreshMyBookingStatus();
  setInterval(expireIfNeeded, 8000);
</script>
</body>
</html>
```

**STAFF'S INTERFACE CODE:**

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width,initial-scale=1"/>
<title>Smart Parking – Gate Verification</title>

<style>
:root{
  --bg:#f6f8fc;
  --card:#ffffff;
  --text:#0f172a;
  --muted:#475569;
  --border:#e2e8f0;
  --green:#16a34a;
  --red:#ef4444;
  --amber:#f59e0b;
  --blue:#2563eb;
}

*{box-sizing:border-box}
body{
  margin:0;
```

```
  font-family:system-ui,-apple-system,Segoe UI,Roboto;
  background:var(--bg);
  color:var(--text);
}

.wrap{
  max-width:800px;
  margin:0 auto;
  padding:20px;
}

header{
  text-align:center;
  margin-bottom:20px;
}

h1{
  margin:0;
  font-size:22px;
}

.sub{
  margin-top:6px;
  color:var(--muted);
```

```
  font-size:13px;
}

.card{
  background:var(--card);
  border:1px solid var(--border);
  border-radius:18px;
  padding:18px;
  box-shadow:0 10px 25px rgba(0,0,0,.05);
}

input{
  width:100%;
  padding:12px;
  border-radius:12px;
  border:1px solid var(--border);
  font-size:15px;
  margin-top:8px;
}

button{
  padding:12px 16px;
  border-radius:12px;
  border:1px solid var(--border);
```

```
  background:white;
  font-weight:700;
  cursor:pointer;
}

button.primary{
  background:var(--blue);
  color:white;
  border-color:var(--blue);
}

button.success{
  background:var(--green);
  color:white;
  border-color:var(--green);
}

button.danger{
  background:var(--red);
  color:white;
  border-color:var(--red);
}

.row{
```

```
  display:flex;
  gap:10px;
  flex-wrap:wrap;
  margin-top:10px;
}

.resultBox{
  margin-top:18px;
  padding:18px;
  border-radius:16px;
  border:2px dashed var(--border);
  text-align:center;
}

.statusBig{
  font-size:28px;
  font-weight:900;
  margin-bottom:8px;
}

.statusValid{
  color:var(--green);
}
```

```
.statusInvalid{
  color:var(--red);
}

.detail{
  margin-top:6px;
  font-size:14px;
  color:var(--muted);
}

.pinBox{
  margin-bottom:20px;
}

.pinStatus{
  font-size:13px;
  margin-top:6px;
}
</style>
</head>

<body>
<div class="wrap">
```

```
<header>
<h1>🅿 Smart Parking – Gate Verification</h1>
<div class="sub">Verify Booking ID and allow vehicle entry</div>
</header>

<div class="card pinBox">
<strong>Staff PIN</strong>
<input type="password" id="staffPin" placeholder="Enter staff PIN">
<div class="row">
<button id="unlockBtn" class="primary">Unlock</button>
</div>
<div id="pinStatus" class="pinStatus"></div>
</div>

<div class="card">
<strong>Enter Booking ID</strong>
<input id="bookingInput" placeholder="Paste booking ID here">

<div class="row">
<button id="checkBtn" class="primary">Verify Booking</button>
<button id="checkinBtn" class="success">Check-In</button>
<button id="cancelBtn" class="danger">Cancel</button>
</div>
```

```
<div class="resultBox" id="resultBox">
<div class="statusBig" id="statusText">Waiting for ID...</div>
<div id="details"></div>
</div>
</div>

</div>

<script type="module">
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.5/firebase-app.js";
import { getDatabase, ref, get, update, remove } from
"https://www.gstatic.com/firebasejs/10.12.5/firebase-database.js";

const firebaseConfig = {
  apiKey: "AIzaSyDTAas1droAkeShhrnvoJhDCc74uJQQhEA",
  authDomain: "esp32-parking-8feef.firebaseapp.com",
  databaseURL: "https://esp32-parking-8feef-default-rtdb.firebaseio.com",
  projectId: "esp32-parking-8feef",
  storageBucket: "esp32-parking-8feef.appspot.com",
  messagingSenderId: "89076263554",
  appId: "1:89076263554:web:61669b435d5c272307e29b",
};

const app = initializeApp(firebaseConfig);
```

```
const db = getDatabase(app);


const el = id => document.getElementById(id);
const nowMs = () => Date.now();


let unlocked = false;
const STAFF_PIN = "1234"; // Change this


el("unlockBtn").onclick = () => {
  if(el("staffPin").value === STAFF_PIN){
    unlocked = true;
    el("pinStatus").innerText = "Access Granted ";
    el("pinStatus").style.color = "green";
  }else{
    el("pinStatus").innerText = "Invalid PIN ";
    el("pinStatus").style.color = "red";
  }
};


async function verifyBooking(){
  if(!unlocked){
    alert("Unlock with staff PIN first.");
    return;
  }
```

```
  const id = el("bookingInput").value.trim();
  if(!id) return;

  const snap = await get(ref(db, "bookings/" + id));

  if(!snap.exists()){
    showInvalid("Booking not found");
    return;
  }

  const data = snap.val();
  if(data.status !== "reserved"){
    showInvalid("Status: " + data.status);
    return;
  }

  showValid(data);
}

function showValid(data){
  el("statusText").innerText = "VALID BOOKING ";
  el("statusText").className = "statusBig statusValid";
  el("details").innerHTML = `
```

```
    <div class="detail">Slot: ${data.slot}</div>
    <div class="detail">Customer: ${data.customerName}</div>
    <div class="detail">Vehicle: ${data.vehicleNo}</div>
  `;
}

function showInvalid(msg){
  el("statusText").innerText = "INVALID ";
  el("statusText").className = "statusBig statusInvalid";
  el("details").innerHTML = `<div class="detail">${msg}</div>`;
}

el("checkBtn").onclick = verifyBooking;

el("checkinBtn").onclick = async () => {
  const id = el("bookingInput").value.trim();
  if(!id) return;

  const snap = await get(ref(db, "bookings/" + id));
  if(!snap.exists()) return;

  const data = snap.val();
  if(data.status !== "reserved"){
    alert("Cannot check-in.");
```

```
    return;
  }

  await update(ref(db, "bookings/" + id), {
    status: "checked_in",
    checkedInAt: nowMs()
  });

  await remove(ref(db, "activeBookings/slot" + data.slot));

  showValid({ ...data, status: "checked_in" });
  alert("Checked-in successfully ");
};

el("cancelBtn").onclick = async () => {
  const id = el("bookingInput").value.trim();
  if(!id) return;

  const snap = await get(ref(db, "bookings/" + id));
  if(!snap.exists()) return;

  const data = snap.val();

  await update(ref(db, "bookings/" + id), {
```

```
    status: "cancelled",
    cancelledAt: nowMs()
  });

  await remove(ref(db, "activeBookings/slot" + data.slot));

  showInvalid("Booking cancelled");
};
</script>

</body>
</html>
```

**MICROCONTROLLER CODE:**

```
#include <WiFi.h>
#include <Firebase_ESP_Client.h>

// WiFi credentials
#define WIFI_SSID "YOUR_WIFI_NAME"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"

// Firebase credentials
#define API_KEY"AIzaSyDTAas1droAkeShhrnvoJhDCc74uJQhEA"
"
#define DATABASE_URL "https://esp32-parking-8feef-default-rtdb.firebaseio.com/"

// IR sensor pins
#define IR1 25
#define IR2 26
#define IR3 27

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

void setup() {
  Serial.begin(115200);

  pinMode(IR1, INPUT);
  pinMode(IR2, INPUT);
  pinMode(IR3, INPUT);

  // WiFi connection
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println("\nWiFi Connected");

  // Firebase configuration
  config.api_key = API_KEY;
  config.database_url = DATABASE_URL;

  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
```

```
}

void loop() {
  int slot1 = digitalRead(IR1);
  int slot2 = digitalRead(IR2);
  int slot3 = digitalRead(IR3);

  // Convert to parking logic
  // 1 = Occupied, 0 = Free
  int status1 = (slot1 == LOW) ? 1 : 0;
  int status2 = (slot2 == LOW) ? 1 : 0;
  int status3 = (slot3 == LOW) ? 1 : 0;

  Firebase.RTDB.setInt(&fbdo, "/parking/slot1", status1);
  Firebase.RTDB.setInt(&fbdo, "/parking/slot2", status2);
  Firebase.RTDB.setInt(&fbdo, "/parking/slot3", status3);

  Serial.println("Data Sent:");
  Serial.print("Slot1: "); Serial.println(status1);
  Serial.print("Slot2: "); Serial.println(status2);
  Serial.print("Slot3: "); Serial.println(status3);

  delay(2000);
}
```